

StarForth Developer Guide

Building, Extending, and Porting the StarForth VM

Robert A. James

Contents

I	Getting Started	1
1	Installation and Build	3
1.1	Installation	3
1.1.1	Prerequisites	3
1.1.2	Building from Source	3
1.1.3	Installation Methods	4
1.1.4	Platform-Specific Notes	4
1.1.5	Verifying the Installation	5
1.1.6	Uninstallation	5
1.1.7	Troubleshooting	5
1.1.8	Building the Documentation	6
1.2	Quick Start	6
1.2.1	Common Build Commands	6
1.2.2	Platform-Specific Builds	6
1.2.3	Performance Modes	7
1.2.4	INIT System	7
1.2.5	Testing and Benchmarking	7
1.2.6	Flags Enabled by <code>make fastest</code>	7
1.2.7	Advanced Build Options	7
1.2.8	Troubleshooting	8
2	Build System Reference	9
2.1	Build Options Reference	9
2.1.1	Performance Optimization Options	9
2.1.2	Architecture Options	10
2.1.3	Platform / Target Options	10
2.1.4	Debugging and Profiling Options	11
2.1.5	Block System Options	11
2.1.6	Memory Configuration Options	11
2.1.7	Feature Flags	12
2.1.8	Build System Variables	12
2.1.9	Build Configuration Matrix	12
2.1.10	Optimization Level Guidelines	12
2.1.11	Usage Examples	13
2.1.12	Compiler Support	13
2.1.13	Performance Tuning Checklist	14
2.1.14	Troubleshooting	14
2.2	Platform Build Guide	14
2.2.1	Platform Abstraction Files	15
2.2.2	L4Re Time Backend	15
2.2.3	Testing	15

2.2.4	Troubleshooting	16
2.3	INIT Tools: Disk-Image Synchronization	16
2.3.1	The Two Core Tools	16
2.3.2	Interactive Wrappers	17
2.3.3	Workflow Patterns	17
2.3.4	The (- Metadata Marker	17
2.3.5	Safety Features	18
2.3.6	Building, Naming, and Testing	18
3	Documentation and API Tooling	19
3.1	The Documentation System	19
3.1.1	Installation Requirements	19
3.1.2	Documentation Targets	19
3.1.3	Documenting Code	20
3.1.4	Coverage Priorities	20
3.1.5	Continuous Integration	20
3.1.6	Output Formats	20
3.1.7	Troubleshooting	21
3.1.8	Customization	21
3.2	Doxygen Style Guide	21
3.2.1	Comment Syntax by Construct	21
3.2.2	Special Tags	23
3.2.3	Quality Guidelines	23
3.2.4	Checking and Coverage	23
3.2.5	IDE Integration	23
3.3	Doxygen Quick Reference	23
3.3.1	Essential Commands	23
3.3.2	Comment Skeletons	24
3.3.3	Common Tags	24
3.3.4	Conditions, Examples, and Grouping	24
3.3.5	Two Templates	24
3.3.6	Best Practices	25
3.3.7	Checking Your Work	25
3.3.8	Effort Estimates	25
II	VM Architecture	27
4	System Overview	29
5	Interpreter and Memory Model	31
5.1	The StarForth Initialization System	31
5.1.1	Components	31
5.1.2	Design Principles	31
5.1.3	init.4th File Format	31
5.1.4	INIT Execution Flow	32
5.1.5	Platform Support	32
5.1.6	The (- Comment Word	33
5.1.7	Dictionary Fence Protection	33
5.1.8	Block Lifecycle	33
5.1.9	Full Boot Sequence	33
5.1.10	Memory Layout	33

5.1.11	LOAD Rewriting Algorithm	34
5.1.12	Performance Characteristics	34
5.1.13	Code References	34
5.1.14	Development Workflow	34
5.1.15	Best Practices	34
6	Physics-Driven Adaptive Runtime	37
7	Pipelining and Speculative Execution	39
8	Heartbeat System	41
8.1	Heartbeat CSV Export	41
8.1.1	CSV Format	41
8.1.2	Capturing Metrics	41
8.1.3	Analysis Use Cases	42
8.1.4	Implementation	42
8.1.5	Known Limitations	43
9	Word-Level ACL	45
10	Capsule System and Mama Forth	47
10.1	Mama Forth: The Multi-VM Supervisor	47
10.1.1	VM-Level Physics Metadata	47
10.1.2	Block Device Physics Metadata	48
10.1.3	Mama’s Responsibilities	48
10.1.4	The Supervision Loop	48
10.1.5	Three-Tier Feedback Loop	49
10.1.6	Worked Example: Thermal Emergency	49
10.1.7	Implementation Phases	49
III	Platform and Hardware	51
11	Platform Abstraction	53
11.1	The Platform Abstraction Layer	53
11.1.1	Structure	53
11.1.2	API	53
11.1.3	Backends	53
11.1.4	Migration from Direct POSIX Calls	54
11.1.5	Benefits	54
11.1.6	StarshipOS Integration and Future Platforms	54
12	Linux and POSIX	55
13	L4Re / Fiasco.OC	57
13.1	L4Re Integration	57
13.1.1	L4Re Fundamentals	57
13.1.2	Package Structure	57
13.1.3	Build System	57
13.1.4	Memory Management via Dataspaces	57
13.1.5	IPC Interface	58
13.1.6	Multi-VM Architecture	58
13.1.7	Kernel-Context Forth	58

13.1.8	StarshipOS, Building, and Operations	58
13.2	Dictionary Allocation on L4Re	58
13.2.1	Why It Matters for L4Re	59
13.2.2	Proposed Approach: Conditional Compilation	59
13.2.3	Scope of Change	59
13.2.4	Current Status	59
13.3	Block I/O Porting Endpoints for L4Re	60
13.3.1	The Stable Northbound API	60
13.3.2	In-Process Dataspace Backend	60
13.3.3	Out-of-Process Block Server Backend	60
13.3.4	Factory Helpers and CLI Semantics	61
13.3.5	Composite Backends	61
13.3.6	Summary	61
14	Raspberry Pi 4 / ARM64	63
14.1	Building StarForth on Raspberry Pi 4	63
14.1.1	Building	63
14.1.2	Architecture Detection and Code Integration	63
14.1.3	Performance Tuning	63
14.1.4	Benchmarking	64
14.1.5	Thermal and Power	64
14.1.6	Storage and Deployment	64
14.1.7	Troubleshooting	64
14.2	ARM64 Assembly Optimizations	64
14.2.1	Architectural Advantages	65
14.2.2	Expected Gains	65
14.2.3	TOS Caching in Practice	65
14.2.4	Raspberry Pi 4 Target	65
14.2.5	Build Configuration	66
14.2.6	Benchmark Results	66
14.2.7	Energy Efficiency	66
14.2.8	Known Limitations	66
14.2.9	Portability and Future Work	67
15	Performance Profiling	69
15.1	The Built-In Word Profiler	69
15.1.1	Quick Start	69
15.1.2	Profiling Levels	69
15.1.3	Reading the Report	69
15.1.4	Hot-Word Analysis	70
15.1.5	Development Workflow	70
15.1.6	Implementation	70
15.1.7	C API	70
15.1.8	Integration with External Tools	71
15.2	x86_64 Assembly Optimizations	71
15.2.1	Performance Impact	71
15.2.2	Enabling the Optimizations	71
15.2.3	Direct-Threaded Inner Interpreter	71
15.2.4	Benchmarking	72
15.2.5	L4Re / StarshipOS Integration	72
15.2.6	Debugging and Correctness	72
15.2.7	Platform Compatibility	72

15.2.8	Safety and Tuning	73
15.3	Profile-Guided Optimization	73
15.3.1	Quick Start	73
15.3.2	PGO Targets	73
15.3.3	Benchmark Comparison	73
15.3.4	The Profiling Workload	74
15.3.5	Profile Data	74
15.3.6	Compiler Flags	74
15.3.7	Troubleshooting	74
15.3.8	Best Practices	74
15.3.9	How PGO Works	75
15.3.10	Build Target Comparison	75
15.3.11	Platform Notes	75
IV	Specifications	77
16	HOLA Analytics Protocol	79
16.1	The HOLA Shared-Memory Protocol (Phase 1)	79
16.1.1	Memory Layout	79
16.1.2	Event Records	79
16.1.3	Channels	80
16.1.4	Command Protocol	80
16.1.5	Synchronization Rules	80
16.1.6	Artefacts	81
17	Formal Verification	83
17.1	VM Formalization Roadmap	83
17.1.1	Objectives	83
17.1.2	Session Layout	83
17.1.3	Development Guidelines	84
17.1.4	Next Actions	84
17.1.5	Multi-Day Proof Checklist	84
17.1.6	VM-First Verification Roadmap	84
17.1.7	Status Snapshot	85
A	Physics Design Notes	87
A.1	The Cone of Influence	87
A.1.1	The Model	87
A.1.2	Three Operational Zones	87
A.1.3	Metrics Inventory	88
A.1.4	The Control Layer: Runtime Knobs	88
A.1.5	The Pub/Sub Gateway: Three Phases	88
A.1.6	The Feedback Loop	89
A.1.7	ML and Statistical Feedback (Future)	89
A.1.8	Open Design Questions	89
A.1.9	Glossary	90
A.2	From Observability to Control	90
A.2.1	The Scheduling System	91
A.2.2	The Memory-Management System	91
A.2.3	Integration Points	92
A.2.4	Implementation Phases	92

A.2.5	Worked Examples	92
A.2.6	Critical Design Decisions	93
A.2.7	Open Questions	93
A.3	Physics Signal Inventory	93
A.3.1	L4Re and Microkernel Signals	93
A.3.2	StarForth VM Signals	94
A.3.3	Op-Amp Signal Flow	94
A.3.4	Messaging and IPC Considerations	95
A.3.5	Host Snapshot Shim and Analytics Heap	95
A.3.6	Conventions and Constraints	95
A.3.7	Immediate Research Tasks	95
B	Messaging Architecture	97
B.1	The StarshipOS Messaging Field	97
B.1.1	Conceptual Model	97
B.1.2	Mathematical Formulation	98
B.1.3	Header and State Extensions	98
B.1.4	Scheduler and Credit Integration	99
B.1.5	Monitoring and Control Loop	99
B.1.6	Implications for the Runtime	99
B.1.7	Future Work	99
B.1.8	Reference Header	100
B.1.9	References	100

Part I

Getting Started

Chapter 1

Installation and Build

1.1 Installation

StarForth builds from source on Linux (x86_64, ARM64) and experimentally on macOS and Windows (WSL2). No binary packages are distributed at this time.

1.1.1 Prerequisites

Required

- GCC ≥ 7.0 or Clang ≥ 6.0
- GNU Make
- glibc (standard builds)

Optional

- texinfo — GNU info documentation (`sudo apt-get install texinfo`)
- dpkg-dev — Debian package generation (`sudo apt-get install dpkg-dev`)
- rpm-build — RPM package generation (`sudo dnf install rpm-build`)

1.1.2 Building from Source

The standard optimized build:

```
1 make fastest
```

This produces the maximum-performance binary with ASM optimisations, direct threading, and LTO enabled. Table 1.1 lists all available build targets.

Target	Description	Performance tier
<code>make fastest</code>	Maximum performance build	Tier 5 (highest)
<code>make pgo</code>	Profile-guided optimisation	Tier 6 (5–15% above fastest)
<code>make all</code>	Standard optimised build	Tier 4
<code>make debug</code>	Debug build with symbols	Tier 1
<code>make minimal</code>	Minimal/embedded build	Tier 3

Table 1.1: StarForth build targets.

1.1.3 Installation Methods

System-Wide Installation (Recommended)

```
1 make fastest
2 sudo make install
```

Installs to the following paths:

- Binary: /usr/local/bin/starforth
- Config: /usr/local/etc/starforth/init.4th
- Man page: /usr/local/share/man/man1/starforth.1
- Documentation: /usr/local/share/doc/starforth/
- Info: /usr/local/share/info/starforth.info

Custom Prefix

```
1 make fastest
2 make install PREFIX=$HOME/.local
```

Debian Package

```
1 make deb
2 sudo dpkg -i ../starforth_1.1.0-1_amd64.deb
```

RPM Package

```
1 make rpm
2 sudo rpm -ivh ~/rpmbuild/RPMS/x86_64/starforth-1.1.0-1.x86_64.rpm
```

1.1.4 Platform-Specific Notes

x86_64 Linux

Standard installation works without modification:

```
1 make fastest
2 sudo make install
```

ARM64 (Raspberry Pi 4 and compatible)

Native build on ARM64:

```
1 make rpi4
2 sudo make install
```

Cross-compile from x86_64:

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make rpi4-cross
3 scp build/starforth user@arm64-device:~/
```

macOS (Experimental)

```

1 xcode-select --install
2 make fastest CC=clang
3 sudo make install

```

Windows (WSL2)

```

1 # Inside WSL2 (Ubuntu):
2 make fastest
3 make install PREFIX=$HOME/.local
4 echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
5 source ~/.bashrc

```

1.1.5 Verifying the Installation

```

1 starforth --version
2 starforth --run-tests
3 starforth

```

Quick functional check:

```

1 starforth -c ':_HELLO_"_Hello, _World!"_CR;_HELLO_BYE'
2 starforth --benchmark 10000

```

1.1.6 Uninstallation

```

1 # From source install:
2 sudo make uninstall
3
4 # From Debian package:
5 sudo dpkg -r starforth
6
7 # From RPM package:
8 sudo rpm -e starforth

```

1.1.7 Troubleshooting

gcc: command not found

```

1 # Debian/Ubuntu:
2 sudo apt-get install build-essential
3
4 # Fedora/RHEL:
5 sudo dnf groupinstall "Development Tools"

```

starforth: command not found after installation

```

1 which starforth
2 export PATH=$HOME/.local/bin:$PATH

```

Cannot find init.4th

```

1 export STARFORTH_INIT=/usr/local/etc/starforth/init.4th

```

1.1.8 Building the Documentation

Generate the PDF manual (LaTeX → PDF):

```
1 make book
```

Generate HTML documentation (single-page and multi-page):

```
1 make book-html
```

1.2 Quick Start

Build the fastest binary for the current platform:

```
1 make fastest
```

1.2.1 Common Build Commands

Command	What it does	When to use
<code>make fastest</code>	Maximum speed build	Production, benchmarking
<code>make fast</code>	Fast without LTO	Development, debugging
<code>make debug</code>	Debug build (-g -O0)	Debugging with GDB
<code>make benchmark</code>	Run full benchmark suite	Performance testing
<code>make help</code>	List all options	Reference

Table 1.2: Common StarForth build commands.

1.2.2 Platform-Specific Builds

x86_64 Linux

```
1 make fastest                # Auto-detects x86_64; builds with ASM
   optimisations
2 ./build/starforth
```

Raspberry Pi 4 (native)

```
1 make fastest                # Auto-detects ARM64; optimised for Cortex
   -A72
2 ./build/starforth
```

Raspberry Pi 4 (cross-compile from x86_64)

Requires `gcc-aarch64-linux-gnu`:

```
1 make rpi4-cross            # Builds ARM64 binary with inline ASM
2 scp build/starforth pi@raspberrypi.local:~/
```

The resulting binary is statically linked and requires no runtime dependencies on the target device.

1.2.3 Performance Modes

Build targets are ranked by output speed:

1. `make pgo` — Profile-Guided Optimisation; 5–15% faster than `fastest`; requires two build passes (3–5 minutes).
2. `make fastest` — Recommended for production; ASM optimisations, direct threading, LTO.
3. `make fast` — ASM + direct threading; no LTO; easier to instrument.
4. `make turbo` — ASM optimisations only; no direct threading.
5. `make all` — Standard build (`-O2`).
6. `make debug` — No optimisations (`-O0`); full debug symbols.

1.2.4 INIT System

At startup StarForth automatically loads `./conf/init.4th`, which defines the foundational word set. No configuration is required:

```
1 ./build/starforth
2 ok>          # Words from init.4th are immediately available
```

`init.4th` defines words, installs a dictionary fence to protect them from `FORGET`, and zeros blocks for user use.

1.2.5 Testing and Benchmarking

```
1 make bench          # Quick benchmark (1 million operations)
2 make benchmark      # Full benchmark suite
3 make test           # Run full test suite (936+ tests)
```

Typical performance for 1 million stack operations:

Build type	x86_64	ARM64 (Raspberry Pi 4)
Debug	~800 ms	~1200 ms
Standard	~250 ms	~380 ms
Fastest	~60 ms	~95 ms
PGO	~50 ms	~80 ms

Table 1.3: StarForth benchmark timings (1M stack operations).

1.2.6 Flags Enabled by `make fastest`

1.2.7 Advanced Build Options

Custom compiler flags:

```
1 make fastest CFLAGS="$(make -s print-cflags) -DCUSTOM_FLAG"
```

Static binary:

```
1 make fastest LDFLAGS="-static -s"
```

Alternate compiler:

```
1 make fastest CC=clang
```

Flag	Effect
-O3	Maximum compiler optimisation
-march=native (x86_64)	Use all host CPU features
-march=armv8-a+crc+simd (ARM64)	ARMv8 with NEON
-DUSE_ASM_OPT=1	Assembly optimisations
-DUSE_DIRECT_THREADING=1	Direct-threaded interpreter
-flto	Link-Time Optimisation
-funroll-loops	Loop unrolling
-finline-functions	Aggressive function inlining
-fomit-frame-pointer	Free register for execution

Table 1.4: Compiler flags active in `make fastest`.

1.2.8 Troubleshooting

Illegal instruction error The host CPU does not support a required optimisation. Use a conservative baseline:

```
1 make fastest CFLAGS="$(BASE_CFLAGS) -O3 -march=x86-64 -DUSE_ASM_OPT=1"
```

Build fails outright Verify the standard build first, then re-enable optimisations incrementally:

```
1 make clean && make all
2 make clean && make fast
3 make clean && make fastest
```

Slow performance Confirm optimisation symbols are present in the binary:

```
1 file build/starforth
2 nm build/starforth | grep vm_push_asm
```

Chapter 2

Build System Reference

2.1 Build Options Reference

StarForth exposes a comprehensive set of build-time configuration options controlling optimizations, feature selection, platform targeting, and debugging. All options are passed as preprocessor defines via `CFLAGS` or as Makefile variables. Architecture detection runs automatically at compile time through `arch_detect.h`.

2.1.1 Performance Optimization Options

`USE_ASM_OPT`

Enables hand-optimized, architecture-specific assembly implementations for performance-critical operations. Type: integer (0 or 1). Default: 0.

Affected routines include:

- `vm_push` / `vm_pop` — data stack operations
- `vm_rpush` / `vm_rpop` — return stack operations
- `vm_add_check_overflow` / `vm_sub_check_overflow` — arithmetic with overflow detection
- `vm_mul` / `vm_div` — multiply and divide
- `vm_strcmp_asm` / `vm_memcpy_asm` — string and memory operations
- `vm_min_asm` / `vm_max_asm` — conditional-move optimizations
- `vm_abs_asm` — absolute value (ARM64)
- `vm_clz` / `vm_ctz` — count leading/trailing zeros (ARM64)
- `vm_prefetch` / `vm_prefetch_stream` — cache prefetching (ARM64)

Typical performance impact: $\approx 20\text{--}25\%$ on `x86_64`; $\approx 15\text{--}20\%$ on ARM64. Enabled automatically by the `fastest`, `fast`, `turbo`, and `pgo` targets.

`USE_DIRECT_THREADING`

Switches the inner interpreter from indirect threading (the FORTH-79 default) to direct threading using GCC computed-goto (`&&label` / `goto *ptr`). In direct-threaded mode each word's code field points directly to native code, eliminating one indirection level per dispatch. Type: integer (0 or 1). Default: 0.

Requirements: GCC or Clang; must be combined with `USE_ASM_OPT=1`. Full support is provided for x86_64 (`vm_inner_interp_asm.h`) and ARM64 (`vm_inner_interp_arm64.h`).

Typical performance impact: $\approx 30\text{--}40\%$ over indirect threading when combined with `USE_ASM_OPT=1`. Enabled by `fastest`, `fast`, and `pgo` targets.

NDEBUG

Standard C99 macro. When defined, all `assert()` calls compile to nothing, and StarForth additionally suppresses debug logging in hot paths. Default: undefined (assertions active). Typical impact: $\approx 5\text{--}10\%$ speedup.

STARFORTH_PERFORMANCE

Experimental flag that trades safety for throughput. Reduces stack-bounds checking in hot paths (DUP, SWAP, etc.) and applies `UNLIKELY()` branch hints, assuming well-formed Forth input.

Caution: stack overflows may go undetected. Use only on code that has been exhaustively tested. Typical additional gain: $\approx 5\text{--}8\%$ over standard `-O3`. Relevant sources: `src/word_source/stack_words.c:5` (DUP), `src/word_source/stack_words.c:117` (SWAP).

2.1.2 Architecture Options

Architecture detection is automatic via `arch_detect.h`; manual overrides are available for cross-compilation scenarios.

ARCH_X86_64

Targets x86-64 (AMD64 / Intel 64). Auto-detected from compiler predefined macros (`__x86_64__`, `_M_X64`, `__amd64__`). Enables x86_64 assembly optimizations, SSE4.2 string instructions, and the x86_64 direct-threading header.

ARCH_ARM64

Targets AArch64 (ARMv8-A). Auto-detected from `__aarch64__`, `_M_ARM64`, and `__arm64__`. Enables NEON SIMD instructions, ARM64-specific instructions (CLZ, CTZ, RBIT, REV), ARM64 assembly optimizations, and ARM64 direct threading.

Native ARM64 build: `make rpi4`. Cross-compile from x86_64: `make rpi4-cross`.

ARCH_NAME

Human-readable string set automatically: `"x86_64"`, `"ARM64"`, or `"Unknown"`. Displayed by `make help`.

On an unrecognized architecture the build emits a compiler warning and falls back to pure-C implementations; no assembly optimizations are loaded.

2.1.3 Platform / Target Options

STARFORTH_MINIMAL

Enables minimal/freestanding build suitable for bare-metal or microkernel environments. When defined: ANSI color codes are suppressed in logging; I/O is reduced to minimal output; the build links with `-nostdlib -ffreestanding`. Enabled by `make minimal` and `make 14re`.

L4RE_TARGET

Targets the L4Re microkernel operating system. Enables the L4Re block-storage backend, ROMFS file access in place of POSIX, and implies `STARFORTH_MINIMAL=1`. Status: partially implemented (ROMFS stub present). See `src/word_source/starforth_words.c:225` and `include/blkio_fa`

2.1.4 Debugging and Profiling Options**DEBUG**

Debug builds compile with `-O0 -g -DDEBUG`: no optimization, full debugging symbols, all assertions enabled, verbose logging throughout. Enabled by `make debug`.

PROFILE_ENABLED

Activates StarForth's built-in profiler. Type: integer (0 or 1). Default: 0. The profiler tracks per-word execution counts, call-graph relationships (caller/callee), stack operation counts, and total instruction counts. Overhead: $\approx 10\text{--}15\%$.

```
1 ./build/starforth --profile 2          # enable at depth 2
2 ./build/starforth --profile-report # print results
```

See `include/profiler.h`.

STRICT_PTR

Controls the Forth address-to-C pointer mapping. Type: integer (0 or 1). Default: 1.

- `STRICT_PTR=1` — Forth addresses are real pointers (`cell_t = uintptr_t`).
- `STRICT_PTR=0` — Forth addresses are indices/offsets (segmented memory models).

Disable only for benchmarking or non-hosted targets. See `src/vm_api.c:202` and `Makefile:37`.

2.1.5 Block System Options

- `BLKCFG_DEFAULT_FBS` — default Forth block size in bytes (default: 1024). See `include/blkcfg.h`.
- `BLKCFG_PATH_MAX` — maximum path length for block-storage files (default: 4096).
- `BLK_FORTH_SYS_RESERVED` — number of system-reserved RAM blocks, hidden from user access (default: 32, blocks 0–31).
- `BLK_DISK_SYS_RESERVED` — number of system-reserved disk blocks (default: 32, PBN 1024–1055).

2.1.6 Memory Configuration Options

- `DATA_STACK_SIZE` — data stack depth in cells (default: 256). See `include/vm.h`.
- `RETURN_STACK_SIZE` — return stack depth in cells (default: 256).
- `STARFORTH_STATE_BYTES` — total state-buffer size for minimal builds (default: 8192). See `src/main.c:40–42`.
- `SF_FC_BUCKETS` — number of forget-chain hash buckets; defined in `vm.h`.

2.1.7 Feature Flags

STARFORTH_ANSI

Enables ANSI escape sequences for terminal control in the block editor (LIST, EDIT): screen clear (`\x1b[2J`), cursor positioning, and colored line numbers. See `src/word_source/editor_words.c:74,188,20`

VM_HAS_CURRENT_ENTRY

Enables the `vm->current_entry` field, which tracks the currently-compiling word. Required for the SEE decompiler, recursive word definitions, and dictionary introspection. Default: defined (enabled).

2.1.8 Build System Variables

- **CC** — C compiler. Default: `gcc`. Also accepts `clang` and `aarch64-linux-gnu-gcc`.
- **CFLAGS** — compiler flags. Extends Makefile `BASE_CFLAGS`.
- **LDFLAGS** — linker flags. Default includes `-Wl,-gc-sections -s -flto=auto -fuse-linker-plugin -static`.
- **MINIMAL** — integer (0 or 1). Setting `MINIMAL=1` enables `STARFORTH_MINIMAL` and adds `-nostdlib -ffreestanding`.
- **ASM** — integer (0 or 1). When set to 1, the build emits `.s` assembly listings alongside `.o` object files.

2.1.9 Build Configuration Matrix

Table 2.1: Common build configurations

Target	USE_ASM_OPT	USE_DIRECT_THREADING	NDEBUG	Opt level	Use case
<code>debug</code>	0	0	No	<code>-O0 -g</code>	Development
<code>all</code>	1	0	No	<code>-O2</code>	Default build
<code>turbo</code>	1	0	Yes	<code>-O3 -flto</code>	Fast, no debug
<code>fast</code>	1	1	Yes	<code>-O3</code>	Fast, readable
<code>fastest</code>	1	1	Yes	<code>-O3 -flto</code>	Maximum speed
<code>pgo</code>	1	1	Yes	<code>-O3 -fprofile-use</code>	Absolute fastest
<code>minimal</code>	0	0	No	<code>-O2</code>	Embedded
<code>l4re</code>	0	0	No	<code>-O2</code>	L4Re microkernel
<code>profile</code>	0	0	No	<code>-O1 -g</code>	Profiling

2.1.10 Optimization Level Guidelines

- **-O0 -g (development)** — fastest compile, slowest runtime ($\approx 10\times$ below optimized); full symbols, all assertions. Use for initial development and crash debugging.
- **-O2 (balanced)** — moderate compile overhead; $3\text{--}4\times$ runtime gain over `-O0`. Default production-test builds.
- **-O3 (high performance)** — aggressive inlining and vectorization; $5\text{--}6\times$ over `-O0`. Production releases.

- **-O3 -flto -march=native -fprofile-use (maximum)** — multi-stage build; 7–8× over -O0. Release and benchmark artifacts. Use `make pgo`.

2.1.11 Usage Examples

Maximum performance native build

```
1 make clean && make pgo
```

Produces a PGO-optimized binary with assembly optimizations, direct threading, and link-time optimization.

Debug build with profiler

```
1 make clean && make profile
2 ./build/starforth --profile 3 --profile-report
```

Cross-compile for Raspberry Pi 4

```
1 make rpi4-cross
2 scp build/starforth pi@raspberrypi.local:~/
```

Produces a statically linked ARM64 binary optimized for Cortex-A72.

Minimal embedded build

```
1 make minimal
```

Produces a freestanding binary with no libc dependencies.

AMD Zen 3 custom build

```
1 make clean
2 make CFLAGS="$(BASE_CFLAGS) -O3 -march=znver3 \
3     -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DNDEBUG" \
4     LDFLAGS="-flto -s"
```

Generate assembly listings

```
1 make asm
2 less build/stack_management.s
```

2.1.12 Compiler Support

- **GCC** — recommended. Minimum GCC 7.0; GCC 11.0+ preferred for improved PGO and ARM64 code generation.
- **Clang** — minimum Clang 10.0; computed-goto and all assembly optimizations supported.
- **ARM64 cross-compilation from x86_64:**

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make CC=aarch64-linux-gnu-gcc rpi4-cross
```

Static linking (`-static`) is recommended for cross-compiled binaries.

2.1.13 Performance Tuning Checklist

For maximum throughput:

- Use `make pgo` (profile-guided optimization).
- Enable `USE_ASM_OPT=1` and `USE_DIRECT_THREADING=1`.
- Define `NDEBUG`.
- Pass `-O3 -flto -march=native`.
- Add `-fno-plt -fno-semantic-interposition` to reduce indirection.
- Add `-funroll-loops -finline-functions` for code expansion.
- Link statically (`-static`) to eliminate PLT overhead.
- Strip symbols (`-s`) to reduce binary size.

Expected result: $\approx 7\text{--}8\times$ faster than a debug build; $\approx 1.2\text{--}1.5\times$ faster than a plain `-O3` build.

2.1.14 Troubleshooting

“Unknown architecture” warning The build falls back automatically to pure-C implementations. Assembly optimizations are disabled. To add support for a new architecture, extend `arch_detect.h`.

Direct threading shows no performance gain Verify that both `USE_DIRECT_THREADING=1` and `USE_ASM_OPT=1` are set, that the target is `x86_64` or `ARM64`, and that the compiler is GCC or Clang. At runtime, `WORDS-INFO` reports whether direct threading is active.

PGO coverage mismatch warnings Run `make clean && make pgo` to force a full clean two-stage rebuild. Stale coverage data from a prior instrumentation pass causes these warnings.

2.2 Platform Build Guide

StarForth builds for both POSIX hosts and the L4Re microkernel from a single codebase, selected by one Makefile switch. The POSIX path is the default; the L4Re path is enabled with `L4RE=1`.

```

1  # POSIX build (Linux/macOS/BSD) -- default
2  make
3  make fastest
4  make pgo
5
6  # L4Re/StarshipOS build
7  make L4RE=1
8
9  # Clean switch between platforms
10 make clean && make L4RE=1

```

File	Purpose	Platform
include/platform_time.h	Abstraction API	All
src/platform/platform_init.c	Platform selector	All
src/platform/time_posix.c	POSIX implementation	POSIX
src/platform/time_l4re.c	L4Re implementation	L4Re

Table 2.2: Platform abstraction source files.

2.2.1 Platform Abstraction Files

The platform layer isolates timing and clock functionality behind a small set of files. The remainder of the VM is untouched by the platform choice.

Only three core files consume the abstraction: `src/log.c` (timestamps via `sf_realtime_ns()` and `sf_format_timestamp()`), `src/profiler.c` (monotonic timing via `sf_monotonic_ns()`), and `src/main.c` (calls `sf_time_init()` at startup). All other VM core files, word implementations, and the block I/O system are unchanged.

2.2.2 L4Re Time Backend

The L4Re RTC server provides hardware RTC access (x86 I/O ports, ARM PL031, I2C chips), nanosecond precision via the CPU timestamp counter, an IPC interface for getting and setting time, and suspend/resume handling that refreshes the offset on wakeup. The C API lives in `librtc.a`:

```

1  l4_uint64_t l4rtc_get_timer(void);
2  int l4rtc_get_offset_to_realtime(l4_cap_idx_t server, l4_uint64_t *
   ns);
3  int l4rtc_set_offset_to_realtime(l4_cap_idx_t server, l4_uint64_t ns
   );

```

StarForth's L4Re backend behaves as follows. Initialization attempts to obtain the "rtc" capability from the L4Re environment, queries the RTC offset by IPC when found, and falls back to a zero offset (epoch time) otherwise. Monotonic time always works without an RTC, reading `l4_kip_clock_ns(l4re_kip())` directly from the Kernel Info Page. Real time returns the RTC offset plus the KIP clock, or zero when no RTC is available. Setting time computes a new offset from the desired time minus uptime and writes it via IPC, which requires write permission on the "rtc" capability.

L4Re's libc already provides working `clock_gettime`, `time`, `localtime`, and `strftime`. The platform abstraction is retained anyway for direct control over the RTC capability, explicit handling of a missing RTC, a portable API across all platforms, and to avoid a libc dependency in minimal builds.

2.2.3 Testing

For the POSIX build, confirm timestamped log output and a non-zero RTC check:

```

1  make clean && make
2  ./build/starforth --log-info

```

For the L4Re build inside the StarshipOS tree, `make l4` selects the L4Re backend automatically; running under `scripts/runos.sh` should show working profiler timing (always available via the KIP clock) and working timestamps, which may report 1970 if no RTC server is present. A syntax-checking script can validate both backends without a full L4Re tree by compiling `time_l4re.c` with `-fsyntax-only` (a fatal error on missing L4Re headers is expected) and compiling the POSIX backend and selector cleanly.

2.2.4 Troubleshooting

- *undefined reference to `sf_monotonic_ns`* — ensure `time_posix.c` and `platform_init.c` are compiled.
- *implicit declaration of `sf_time_init`* — add `#include "platform_time.h"`.
- *fatal error: `l4/re/env.h`: No such file or directory* — expected outside the L4Re tree; build through the StarshipOS build system.
- *RTC server not found* — start the RTC server in the loader script and provide the `rtc` capability.
- *Timestamps show 1970-01-01* — the RTC server is not supplying an offset; check the vbus configuration and hardware RTC.

The result is a complete platform abstraction layer that builds on POSIX and L4Re alike, with zero runtime overhead from inline functions and a clean vtable-style separation. Switching platforms requires only `make L4RE=1`; the core VM is unchanged.

2.3 INIT Tools: Disk-Image Synchronization

StarForth ships a toolchain that maintains a bidirectional flow between `./conf/init.4th` — the version-controlled source of truth — and the disk images (`.img` files) tracked through Git LFS. The arrangement supports collaborative development: developers edit blocks natively inside a disk image using LIST, EDIT, and friends; tools extract the annotated blocks into `init.4th`; those changes are reviewed in Git pull requests; and approved changes are applied back to disk images.

2.3.1 The Two Core Tools

extract-init

`tools/extract-init` (C) scans a disk image for blocks marked with a `(- ... )` header and writes them to an `init.4th` file.

```
1 ./tools/extract-init --img=disks/example.img --out=conf/init-4.4th \
2   [--fbs=1024] [--start=0] [--end=-1] \
3   [--loose] [--require-close] [--max=N]
```

- `-img=PATH` — disk image to scan (required).
- `-out=PATH` — output `init.4th` file (required).
- `-fbs=N` — Forth block size in bytes (default 1024).
- `-start=N`, `-end=N` — block range; `-1` reads to end of file.
- `-loose` — permit a BOM or leading whitespace before the `(-` header.
- `-require-close` — require the closing `)` in the same block.
- `-max=N` — stop after extracting `N` blocks.

The detection algorithm reads each block sequentially, checks for `(-` at the start (tolerating a BOM or whitespace under `-loose`), optionally verifies a closing `)` when `-require-close` is set, and extracts matching blocks. Output is a header section followed by `Block N` segments.

apply-init

`tools/apply-init` (C) parses an `init.4th` file and writes its `Block N` sections to the corresponding blocks of a disk image.

```
1 ./tools/apply-init --img=disks/example.img --in=conf/init-4.4th \
2   [--fbs=1024] [--start=0] [--end=N] \
3   [--clip] [--dry-run] [--verify] [--verbose]
```

Only lines between a `Block N` marker and the next marker (or EOF) are written to block *N*; everything outside a block section, including top-level comments, is ignored. The `-start` and `-end` options act as write guards, `-clip` truncates oversized blocks, `-dry-run` prints the plan without writing, and `-verify` re-reads each block after writing for a byte-for-byte comparison.

2.3.2 Interactive Wrappers

Two shell wrappers add interactive prompts over the C tools. `scripts/update-init.sh` lists the disk images under `./disks/`, prompts for an image and extraction parameters, displays the command, and runs `extract-init` after confirmation — the natural step after editing blocks in an image. `scripts/apply-init.sh` mirrors this for `apply-init`, used after merging a pull request that changed `init.4th`.

2.3.3 Workflow Patterns

Extract from disk to Git. After working in a disk image, extract the marked blocks, review the diff, and commit:

```
1 ./scripts/update-init.sh           # select disk, confirm extraction
2 git diff conf/init-4.4th
3 git add conf/init-4.4th
4 git commit -m "Add new INIT words: FOO and BAR"
```

Apply from Git to disk. After pulling a merged change, apply it to the local image and verify in the REPL:

```
1 git pull origin master
2 ./scripts/apply-init.sh           # select disk, confirm application
```

Automation. For CI/CD the C tools run unattended with the full set of flags, for example an apply guarded to blocks at or above 1024, clipping oversized content, with read-back verification and verbose output.

2.3.4 The (- Metadata Marker

The `(-` word is dual-purpose: at runtime it behaves as a Forth comment (like `(` but with a dash), and to the tooling it marks an extraction-worthy block.

```
1 Block 2048
2 (- String utilities: COUNT, COMPARE, SEARCH )
3 : COUNT DUP 1+ SWAP C@ ;
4 : COMPARE ... ;
```

The dash distinguishes deliberate metadata from the ordinary `(` comments that fill normal blocks, avoids false positives, and is trivially greppable with `grep "\(-"`. The implementation lives in `src/word_source/starforth_words.c` and is registered in both the FORTH and STARFORTH vocabularies:

```

1 void starforth_word_paren_dash(VM *vm) {
2     /* Consume input from "(- " to first ")" */
3     int depth = 1;
4     while (vm->input_pos < vm->input_length && depth > 0) {
5         char c = vm->input_buffer[vm->input_pos++];
6         if (c == '(') depth++;
7         else if (c == ')') depth--;
8     }
9     if (depth > 0) {
10        log_message(LOG_WARN, "(-_comment_not_terminated");
11    }
12    log_message(LOG_DEBUG, "(-_comment_parsed_(init.4th_metadata_
13        marker)");
14 }

```

2.3.5 Safety Features

Four mechanisms protect against accidental data loss. *Guard rails* (`-start / -end`) refuse writes outside an allowed block range, protecting the boot area (blocks 0–1023) and system ranges. *Verification* (`-verify`) seeks back, re-reads the full block, and compares it against what was written, catching silent write failures. *Dry run* (`-dry-run`) prints the write plan without touching the disk. *Content-length checks* reject blocks longer than the Forth block size by default; `-clip` truncates instead (not recommended for code).

2.3.6 Building, Naming, and Testing

The interactive scripts rebuild the binaries when the source is newer; manual builds use a C99 compiler with POSIX 64-bit file offsets:

```

1 gcc -std=c99 -O2 -Wall -Wextra -Werror \
2     -o tools/extract-init tools/extract_init.c
3 gcc -std=gnu99 -D_FILE_OFFSET_BITS=64 -O2 -Wall -Wextra -Werror \
4     -o tools/apply-init tools/apply_init.c

```

Disk images follow the convention `<hostname>-<username>-<version>.img`; images go to Git LFS while `conf/init.4th` stays in plain Git as a diffable text file. A round-trip test — apply `init.4th` to a disk, extract it back, and diff (ignoring the metadata header) — should reproduce the original. The block size must match across extract, apply, and the VM's expectation.

Chapter 3

Documentation and API Tooling

3.1 The Documentation System

StarForth generates its API documentation directly from source, driven by Doxygen and rendered into several formats. A single command builds everything:

```
1 make docs
```

The build emits AsciiDoc API documentation under `docs/src/appendix/`, one flat file per source module. These files are produced on demand by `./scripts/generate-doxygen-appendix.sh` and rebuilt periodically by the Jenkins pipeline. To inspect the result, run the generator and read the index:

```
1 ./scripts/generate-doxygen-appendix.sh
2 cat docs/src/appendix/index.adoc
```

3.1.1 Installation Requirements

Core functionality requires Doxygen and Graphviz. On the major distributions:

```
1 sudo apt-get install doxygen graphviz      # Ubuntu/Debian
2 brew install doxygen graphviz              # macOS
3 sudo dnf install doxygen graphviz          # Fedora/RHEL
```

PDF output additionally requires a TeX Live installation (`texlive-latex-base` and `texlive-latex-extra` on Debian, or MacTeX/BasicTeX on macOS). AsciiDoc and Markdown conversion requires Pandoc, with Asciidoctor optional for HTML rendering. Verify the toolchain before building — Doxygen should report version 1.9.0 or later:

```
1 doxygen --version
2 dot -V
3 pandoc --version      # optional
4 pdflatex --version    # optional
```

3.1.2 Documentation Targets

Target	Description	Requirements
<code>make docs</code>	Generate all formats	doxygen, graphviz (+ optional)
<code>make docs-html</code>	HTML only (fast)	doxygen, graphviz
<code>make docs-pdf</code>	PDF only	doxygen, graphviz, pdflatex
<code>make docs-open</code>	Generate HTML and open browser	doxygen, graphviz
<code>make docs-clean</code>	Remove generated docs	none

3.1.3 Documenting Code

Documentation lives beside the code it describes. The workflow is short: read the style guide, copy the template at `examples/doxygen_example.h`, then annotate. A typical function comment carries a brief, a detailed body, parameters, contracts, and an example:

```

1  /**
2   * @brief Push value onto data stack
3   *
4   * @details
5   * Adds a value to the top of the data stack. Stack overflow
6   * is checked and vm->error is set if stack is full.
7   *
8   * @param vm Pointer to VM instance
9   * @param value Value to push
10  *
11  * @pre vm must be initialized
12  * @pre vm->dsp < STACK_SIZE-1
13  * @post vm->dsp incremented by 1
14  * @post On error: vm->error is set
15  *
16  * @note This is a hot-path function - optimized for speed
17  * @warning Always check vm->error after calling
18  *
19  * @see vm_pop()
20  */
21 void vm_push(VM *vm, cell_t value);

```

After annotating, rebuild and clear any warnings:

```

1  make docs-html
2  cat docs/src/appendix/doxygen_warnings.log

```

3.1.4 Coverage Priorities

The target is 100% coverage of the public API headers. Documentation effort is tiered by audience:

- **High priority** — user-facing API: `include/vm.h`, `include/word_registry.h`, `include/log.h`, `include/io.h`.
- **Medium priority** — developer API: word implementation headers under `src/word_source/include/`, `include/profiler.h`, `include/vm_debug.h`.
- **Low priority** — internal implementation files and test infrastructure headers.

3.1.5 Continuous Integration

The documentation build is CI-friendly. A GitHub Actions job installs Doxygen and Graphviz, runs `make docs-html`, fails the build if `doxygen_warnings.log` is non-empty, and deploys the generated HTML to GitHub Pages on the master branch.

3.1.6 Output Formats

Each format serves a distinct audience. HTML is best for interactive browsing and search; PDF for printing and offline reference; AsciiDoc for downstream conversion; Markdown for GitHub and wikis; and man pages for command-line reference, installable into the system man path.

3.1.7 Troubleshooting

The common failures are missing tools. A `command not found` for `doxygen`, `dot`, or `pdflatex` means the corresponding package is absent — install it, or fall back to `make docs-html` since PDF and Pandoc-based outputs are optional. Missing graphs indicate Graphviz is not installed. Warnings about undocumented functions are resolved by annotating the offending code per the style guide. “No such file or directory” when opening docs means they have not been generated yet.

3.1.8 Customization

Edit `Doxyfile` to adjust the project name and version (`PROJECT_NAME`, `PROJECT_NUMBER`), input file set (`INPUT`, `FILE_PATTERNS`), output formats, and diagram options (`CALL_GRAPH`, `CALLER_GRAPH`). Custom Markdown pages are added by appending them to `INPUT`. HTML appearance is themed through `HTML_EXTRA_STYLESHEET` and `HTML_COLORSTYLE`.

External projects can link against StarForth documentation by referencing the generated tag file `docs/src/appendix/starforth.tag` in their own `Doxyfile` via `TAGFILES`.

3.2 Doxygen Style Guide

StarForth generates its API documentation from Javadoc-style comments embedded in the source. This guide defines the house style for those comments. Doxygen renders them into HTML, PDF, AsciiDoc, Markdown, and Unix man pages; a single comment therefore feeds every audience, so it pays to write each one with care. Generate with `make docs` for all formats, `make docs-html` for a fast HTML-only pass, or `make docs-open` to build and open in a browser.

3.2.1 Comment Syntax by Construct

File Headers

Every header and source file opens with an `@file` block carrying a brief, a detailed description, and provenance:

```

1  /**
2   * @file vm.h
3   * @brief StarForth Virtual Machine Core API
4   *
5   * @details
6   * Detailed description of what this file contains and its purpose.
7   *
8   * @author R. A. James (rajames)
9   * @date 2025-08-15
10  * @version 1.0.0
11  * @copyright CC0-1.0 (Public Domain)
12  */

```

Functions

Functions carry `@brief`, `@param`, `@return`, and an optional `@details`. Substantial functions also document their contracts (`@pre`, `@post`), call out hazards with `@note` and `@warning`, cross-reference relatives with `@see`, and supply a runnable example:

```

1  /**
2   * @brief Initialize the StarForth virtual machine
3   *

```

```

4  * @details
5  * Allocates VM memory, initializes stacks, sets up the dictionary,
6  * and registers the FORTH-79 standard word set.
7  *
8  * @param vm Pointer to uninitialized VM structure
9  *
10 * @pre vm must point to valid memory
11 * @post vm->memory is allocated (VM_MEMORY_SIZE bytes)
12 * @post vm->error is 0 on success, 1 on failure
13 *
14 * @warning Do not use the VM if vm->error is non-zero after init
15 * @see vm_cleanup()
16 *
17 * @par Example:
18 * @code
19 * VM vm;
20 * vm_init(&vm);
21 * if (vm.error) return 1;
22 * vm_interpret(&vm, "42 . CR");
23 * vm_cleanup(&vm);
24 * @endcode
25 */
26 void vm_init(VM *vm);

```

Types, Structures, and Members

Typedefs and structures take `@typedef` or `@struct` with a brief and details; each member is documented inline. Note platform caveats and thread safety where relevant — the VM structure, for instance, is documented as *not* thread-safe, requiring one instance per thread or external locking.

```

1  /**
2  * @typedef cell_t
3  * @brief Forth cell type (signed 64-bit integer)
4  *
5  * @note Size is platform-dependent: sizeof(signed long)
6  */
7  typedef signed long cell_t;
8
9  typedef struct VM {
10     /** @brief Data stack (1024 cells) */
11     cell_t data_stack[STACK_SIZE];
12
13     /**
14      * @brief Data stack pointer (index of top element)
15      * @details dsp == -1 means empty; STACK_SIZE-1 means full
16      */
17     int dsp;
18 } VM;

```

Enums and Macros

Enumerations use `@enum` with inline member documentation; macros use `@def`. Document the meaning and any layout consequence — for example, `VM_MEMORY_SIZE` is annotated with the 2 MB dictionary / 3 MB user block split.

3.2.2 Special Tags

Related functions are organized with `@defgroup` and `@ingroup`, bracketed by `@{` and `@}`. Cross-references use `@see`; examples use `@code` / `@endcode`. Contract and lifecycle tags — `@pre`, `@post`, `@note`, `@warning`, `@bug`, `@todo`, `@deprecated` — carry the operational caveats a caller must know.

3.2.3 Quality Guidelines

Write documentation that earns its place:

- Document every public API function; supply examples for the complex ones; state pre- and post-conditions; flag dangerous operations with `@warning`; cross-reference with `@see`; keep `@brief` to a single line and push depth into `@details`.
- Do not document private static functions unless they are genuinely intricate, restate the function name, write obvious comments (“`@brief Get value`” for `getValue()`), use vague phrasing, or let the comment drift out of sync with the code.

A complete worked header lives at `examples/doxygen_example.h`.

3.2.4 Checking and Coverage

After annotating, rebuild and clear the warning log:

```
1 make docs-html
2 cat docs/api/doxygen_warnings.log
```

The frequent offenders are undocumented functions, missing `@param` entries, missing `@return` on non-void functions, and broken `@see` targets. The coverage goal is 100% across the public headers in `include/`, the word-source headers in `src/word_source/include/`, and the key implementation files in `src/`.

3.2.5 IDE Integration

VS Code generates templates through the “Doxygen Documentation Generator” extension; CLion and IntelliJ have built-in support triggered by typing `/**` and Enter; Vim uses the DoxygenToolkit plugin.

3.3 Doxygen Quick Reference

A one-page cheat sheet for annotating StarForth code. The full conventions live in the Doxygen Style Guide; this card is the working reference.

3.3.1 Essential Commands

```
1 make docs           # all formats (HTML, PDF, AsciiDoc, MD, man)
2 make docs-html      # HTML only (fastest)
3 make docs-open      # generate and open in browser
4 make docs-clean     # remove generated docs
```

3.3.2 Comment Skeletons

A file header carries `@file`, `@brief`, `@author`, and `@date`. A function carries `@brief`, one `@param` per argument, and `@return`. A struct uses `@struct` with inline member comments (`/**< ... */`); an enum uses `@enum` likewise; a macro uses `@def`; a typedef uses `@typedef`.

```

1  /**
2   * @brief One-line description
3   * @param name Parameter description
4   * @return Return value description
5   */
6  type function(type name);

```

3.3.3 Common Tags

Tag	Purpose	Example
<code>@brief</code>	Short description	<code>@brief Initialize VM</code>
<code>@details</code>	Detailed description	<code>@details Allocates memory...</code>
<code>@param name</code>	Parameter	<code>@param vm VM instance pointer</code>
<code>@return</code>	Return value	<code>@return 0 on success</code>
<code>@retval value</code>	Specific return	<code>@retval 0 Success</code>
<code>@see</code>	Cross-reference	<code>@see vm_cleanup()</code>
<code>@note</code>	Important note	<code>@note Thread-safe</code>
<code>@warning</code>	Warning	<code>@warning May block</code>
<code>@bug</code>	Known bug	<code>@bug Issue #42</code>
<code>@todo</code>	Future work	<code>@todo Add optimization</code>
<code>@deprecated</code>	Deprecated	<code>@deprecated Use foo() instead</code>

3.3.4 Conditions, Examples, and Grouping

Invariants use `@pre`, `@post`, and `@invariant`. Examples sit inside `@code` / `@endcode` under a `@par Example:`. Related functions are bracketed by `@defgroup ... @{` and `@}`. Within comments, Markdown-style formatting works: hyphen bullets and numbered lists for sequences, `##` headings for sections, and `*italic*`, `**bold**`, and `'code'` for emphasis.

3.3.5 Two Templates

A simple function needs only brief, params, and return. A complex one earns the full treatment:

```

1  /**
2   * @brief Short description
3   *
4   * @details
5   * Detailed explanation of what this function does and why.
6   *
7   * @param vm VM instance pointer
8   * @param value Input value
9   *
10  * @return Result value
11  * @retval 0 Success
12  * @retval -1 Error
13  *
14  * @pre vm must be initialized
15  * @post vm->state is updated
16  *

```

```

17  * @warning Potential issue to be aware of
18  * @see related_function()
19  *
20  * @par Example:
21  * @code
22  * int result = my_func(&vm, 42);
23  * if (result < 0) handle_error();
24  * @endcode
25  */
26  int my_func(VM *vm, int value);

```

3.3.6 Best Practices

Document every public function, keep the brief to one line, push depth into `@details`, supply examples for complex functions, cross-reference with `@see`, document all parameters and return values, and flag hazards with `@warning` and `@note`. Do not document the obvious, restate the function name, omit parameter or return descriptions, write vague prose, use opaque names in examples, or let documentation rot when the code changes.

3.3.7 Checking Your Work

Generate, inspect the warning log, then view:

```

1  make docs-html
2  cat docs/api/doxygen_warnings.log
3  make docs-open

```

Warning	Fix
"Member X is not documented"	Add <code>@param X description</code>
"No documentation for function"	Add <code>@brief</code> comment
"Return value not documented"	Add <code>@return description</code>
"Invalid cross-reference"	Check the <code>@see</code> target exists

3.3.8 Effort Estimates

A simple function takes 2–5 minutes; a complex function with an example, 10–15; a struct with ten fields, 10–15; and a complete 20-function header, one to two hours. IDE template generators are available for VS Code (the “Doxygen Documentation Generator” extension), CLion (built in), and Vim (the DoxygenToolkit plugin with `:Dox`).

Part II

VM Architecture

Chapter 4

System Overview

Chapter 5

Interpreter and Memory Model

5.1 The StarForth Initialization System

StarForth loads foundational Forth definitions from `./conf/init.4th` at startup. The system cleanly separates boot-time initialization from runtime operation, zeroing all initialization blocks once execution completes and protecting the resulting dictionary words from accidental removal.

5.1.1 Components

- `./conf/init.4th` — version-controlled initialization file containing block-structured Forth source.
- **INIT word** — core initialization primitive, implemented in `src/word_source/starforth_words.c`.
- **(- comment word** — dual-purpose: a standard Forth comment at runtime and a tooling metadata marker.
- **Dictionary fence** — prevents `FORGET` from removing any word defined during initialization.
- **Block subsystem** — provides transient block storage for initialization code.

5.1.2 Design Principles

- **Deterministic boot.** The same `init.4th` always produces the same VM state.
- **Transient initialization.** Blocks execute then vanish; their storage is zeroed and returned to userspace.
- **Protected dictionary.** Initialization words cannot be removed with `FORGET`.
- **No runtime dependencies.** After boot the VM has no file system dependencies.
- **Platform agnostic.** The same mechanism works with both filesystem (Linux) and ROMFS (L4Re) backends.

5.1.3 `init.4th` File Format

Each file consists of one or more numbered blocks. Each block opens with a **Block <n>** header line and, conventionally, a `(- comment` describing its purpose.

```

1 Block 2048
2 (- Large Letter F )
3 : STAR 42 EMIT ;
4 : STARS 0 DO STAR LOOP ;
5 : MARGIN CR 30 SPACES ;
6 : BLIP MARGIN STAR ;
7 : BAR MARGIN 5 STARS ;
8 : F BAR BLIP BAR BLIP BLIP ;
9
10 Block 3000
11 (- Add Me to init.4th )
12 : E BAR BLIP BAR BLIP BAR ;
13
14 Block 3001
15 (- Add Me to init.4th )
16 E F CR

```

Format rules:

- Each block starts with **Block <number>**.
- Blocks load in file order (1, 2, 3, ...) regardless of original block numbers.
- **LOAD** references such as 2048 **LOAD** are automatically remapped to sequential numbers.
- Newlines within blocks are preserved for correct parsing.
- The **->** word must not appear in **init.4th**; use separate blocks instead to avoid double execution.

5.1.4 INIT Execution Flow

1. Read `./conf/init.4th` (or ROMFS on L4Re).
2. Parse block headers and build a mapping table (**Block 2048** → sequential 1, etc.).
3. Copy block content sequentially, rewriting **LOAD** references to their sequential equivalents.
4. Execute all blocks via **LOAD**.
5. Switch to the **FORTH** vocabulary context.
6. Zero all initialization blocks to reclaim them for userspace.
7. Return to `main.c`, which immediately sets the dictionary fence.

Stack effect: **INIT** (--).

INIT is fatal if `init.4th` cannot be opened, if a block allocation fails, or if a block fails to execute. The system cannot enter the **REPL** until initialization completes successfully.

5.1.5 Platform Support

On Linux, **INIT** opens the file with `fopen`:

```
1 FILE *fp = fopen("./conf/init.4th", "r");
```

On L4Re, the file will be read from a ROMFS dataspace declared in `modules.list`.

5.1.6 The (- Comment Word

The (- word consumes input from the opening marker to the first unmatched closing parenthesis, handling nested parentheses correctly. It has no stack effect and logs at `DEBUG` level. Tooling in `tools/` uses it as a metadata extraction marker; the runtime treats it as an ordinary comment.

5.1.7 Dictionary Fence Protection

After `INIT` returns, `main.c` sets the dictionary fence:

```

1  /* Set dictionary fence after INIT to protect foundational words
2     from FORGET */
3  vm.dict_fence_latest = vm.latest;
4  vm.dict_fence_here   = vm.here;
5  log_message(LOG_INFO,
6             "Dictionary fence set - init words protected from FORGET");

```

Any subsequent `FORGET` that would reach below this fence silently fails, preventing users from destabilizing words that other definitions depend on.

5.1.8 Block Lifecycle

During INIT: Blocks 1–N hold the parsed content of `init.4th`.

After INIT: Blocks 1–N are zeroed. The dictionary retains all defined words, all protected by the fence.

Runtime: Blocks 1–992 are available to userspace programs as a clean slate with no initialization dependency.

5.1.9 Full Boot Sequence

1. Parse command-line arguments.
2. Open the block device (file or RAM backend).
3. Initialize the VM (`vm_init`).
4. Initialize the block subsystem (1 MB RAM blocks 0–1023).
5. Register all FORTH-79 words.
6. Run the `INIT` word.
7. Set the dictionary fence.
8. Start the REPL.

5.1.10 Memory Layout

VM memory (5 MB) Dictionary grows upward: system words, then initialization words (all protected by the fence), then user words.

Block RAM (1 MB) Block 0 is reserved for volume metadata. Blocks 1–992 are user blocks, zeroed after `INIT`. Blocks 993–1023 are reserved.

5.1.11 LOAD Rewriting Algorithm

The second pass of INIT walks the copied block content and rewrites any NNNN LOAD pattern found in it:

```

1  /* First pass: build mapping table
2     Block 2048 -> Sequential 1
3     Block 3000 -> Sequential 2
4     Block 3001 -> Sequential 3 */
5
6  /* Second pass: copy with rewriting */
7  while (copying block content) {
8      if (found "NNNN_LOAD" pattern) {
9          lookup NNNN in mapping;
10         replace with mapped number;
11         write "M_LOAD" to block;
12     }
13 }
```

5.1.12 Performance Characteristics

Initialization completes in under 10 ms for a typical `init.4th`. The transient blocks are zeroed immediately after execution, so the memory overhead is zero at runtime. Dictionary size grows proportionally to the number of definitions in `init.4th`.

5.1.13 Code References

src/word_source/starforth_words.c:214–495 INIT word implementation. File open (234–240), block mapping (272–305), block copy with LOAD rewriting (307–418), block execution (447–465), vocabulary switch (468–481), block zeroing (484–493).

src/main.c:488–491 Dictionary fence setting.

src/word_source/defining_words.c:530–612 FORGET implementation; fence check at lines 559–571.

src/word_source/block_words.c:191–213 LOAD implementation.

src/word_source/block_words.c:289–309 -> implementation (block continuation).

5.1.14 Development Workflow

In IDE mode (Linux), edit `./conf/init.4th` directly and rebuild. The file is plain text; diffs are readable in code review.

In the L4Re deployment path, `init.4th` is packed into a ROMFS image at build time and declared as a module in `modules.list`. The resulting binary carries no filesystem dependency at runtime.

5.1.15 Best Practices

- Keep `init.4th` minimal — only foundational definitions.
- Always annotate blocks with (– comments.
- Define dependencies before the words that use them; avoid forward references.
- Add blocks incrementally and test after each addition.

- Avoid side effects in initialization blocks; the last block may optionally invoke a self-test.

Chapter 6

Physics-Driven Adaptive Runtime

Chapter 7

Pipelining and Speculative Execution

Chapter 8

Heartbeat System

8.1 Heartbeat CSV Export

The heartbeat subsystem emits real-time multivariate VM metrics to `stderr` in CSV form, exposing StarForth's adaptive runtime to external analysis. Each row carries eleven performance indicators sampled once per heartbeat tick — nominally every millisecond — producing a continuous time series of the running system's internal state.

The design favours streaming over buffering. Metrics are read directly from VM state with no intermediate copy, each row is flushed immediately so that downstream tools see data as it is produced, and the stream is confined to `stderr` to keep `stdout` reserved for test output. The emitting thread is a background `pthread` decoupled from the main interpreter, so capture does not perturb execution timing beyond a fixed, small cost per tick.

8.1.1 CSV Format

The VM emits raw rows only; column names are supplied separately by analysis tooling. The reference header is:

```
1 tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,
   bucket_hits_delta,
2 word_executions_delta,hot_word_count,avg_word_heat>window_width,
3 predicted_label_hits,estimated_jitter_ns
```

The *Status* column below reflects which fields carry live values and which currently emit a placeholder zero pending delta-tracking work.

8.1.2 Capturing Metrics

The simplest capture redirects `stderr` to a file and discards `stdout`:

```
1 # Capture metrics, discard test output
2 ./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null
3
4 # Keep both streams, separated
5 ./build/amd64/fastest/starforth --doe 1>test_output.txt 2>heartbeat.
   csv
```

Because rows are flushed as they are produced, the stream can be monitored live — for example piped through `csvlook` from `csvkit`, or followed with `tail -f`. For post-processing, prepend the reference header and load the result into R or pandas:

```
1 { echo "tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,\
2 bucket_hits_delta,word_executions_delta,hot_word_count,avg_word_heat
   ,\
```

Table 8.1: Heartbeat CSV columns

Column	Type	Unit	Description	Status
<code>tick_number</code>	<code>uint32</code>	count	Monotonic tick counter since VM start	live
<code>elapsed_ns</code>	<code>uint64</code>	ns	Time since VM initialization	live
<code>tick_interval_ns</code>	<code>uint64</code>	ns	Time since previous tick (actual vs. nominal 1 ms)	live
<code>cache_hits_delta</code>	<code>uint32</code>	count	Hotwords cache hits since last tick	TODO
<code>bucket_hits_delta</code>	<code>uint32</code>	count	Bucket-level cache hits since last tick	TODO
<code>word_executions_delta</code>	<code>uint32</code>	count	FORTH word executions since last tick	TODO
<code>hot_word_count</code>	<code>uint64</code>	count	Words at or above the heat promotion threshold (≥ 10)	live
<code>avg_word_heat</code>	<code>double</code>	heat	Mean execution heat across the dictionary ($\mathbb{Q}_{48.16} \rightarrow \text{double}$)	live
<code>window_width</code>	<code>uint32</code>	entries	Current rolling window effective size (adaptive)	live
<code>predicted_label_hits</code>	<code>uint32</code>	count	Successful pipelining speculation hits	TODO
<code>estimated_jitter_ns</code>	<code>double</code>	ns	Absolute deviation from the nominal 1 ms interval	live

```

3 window_width,predicted_label_hits,estimated_jitter_ns"; \
4 cat heartbeat.csv; } > analysis.csv

```

8.1.3 Analysis Use Cases

The eleven-dimensional series supports three broad classes of analysis. For dynamics modeling, the trajectory admits phase-space reconstruction to locate attractor basins in the adaptive parameter space, Lyapunov-exponent estimation to quantify stability, and mutual-information analysis to discover coupling between metrics. For tuning validation, window-width adaptation can be correlated against prediction accuracy, heat-decay behaviour against stale word accumulation, and jitter against thread-scheduling effects. For workload profiling, the same data yields hot-word churn rate, execution density (`word_executions_delta` over `tick_interval_ns`), and cache efficiency relative to working-set size.

8.1.4 Implementation

Capture is performed by `heartbeat_capture_tick_snapshot()`, which reads the monotonic tick counter, computes elapsed and inter-tick time, walks the dictionary to count hot words and their mean heat, samples the rolling window size, and estimates jitter as the absolute difference between the actual and nominal interval. The walk is linear in dictionary size (on the order of a few hundred words). Emission is performed by `heartbeat_emit_tick_row()`, which formats the eleven values and flushes `stderr` immediately. Both are invoked once per cycle from the background heartbeat thread.

The thread runs without the original 50 ms startup delay, so metrics begin flowing immediately even for short-running DoE workloads. The trade-off is that the first few ticks may show transient values during the word-registration phase.

8.1.5 Known Limitations

Three groups of fields are not yet wired to live deltas and currently emit zero: the cache, bucket, and word-execution counters all require per-tick baseline tracking in the heartbeat state. Separately, the source notes an uninitialized `run_start_ns`, which can produce implausibly large `elapsed_ns` values in the first ticks until the field is seeded at VM initialization, and a missing aggregation of per-word prefetch hits behind `predicted_label_hits`.

The runtime cost is small: emission adds well under one percent CPU at the nominal tick rate, the snapshot is a stack-allocated structure of under a hundred bytes, and the output volume is on the order of a hundred kilobytes per second of execution.

Chapter 9

Word-Level ACL

Chapter 10

Capsule System and Mama Forth

10.1 Mama Forth: The Multi-VM Supervisor

Mama Forth is the governing FORTH VM that orchestrates a multi-VM ecosystem. It departs from conventional supervisor design in one decisive respect: Mama Forth is itself a FORTH VM, a first-class citizen carrying the same physics instrumentation as every VM it governs. The supervisor does not stand outside the runtime model; it participates in it.

The ecosystem forms a tree. Mama Forth sits at the root. Beneath it, child VMs each carry their own physics state. Beneath each child, block devices carry per-device physics. Mama observes all three layers and makes system-level decisions: when to spawn or kill a child VM, how to allocate block devices to children, how to enforce governance across the ecosystem, and how to respond to emergencies such as thermal runaway or memory pressure.

The central design insight is that physics metadata becomes a three-tier hierarchy:

- **Word level** — per-word metrics: entropy, temperature, latency.
- **VM level** — per-VM aggregates: child health, resource usage.
- **Block level** — per-block device metrics: I/O patterns, thermal distribution.

10.1.1 VM-Level Physics Metadata

Today's physics model tracks per-word metrics attached to each `DictEntry`:

```
1 typedef struct {
2     uint16_t temperature_q8;    // Thermal state [0x0000, 0xFFFF]
3     uint64_t last_active_ns;    // Last execution timestamp
4     uint32_t entropy;           // Execution randomness
5     int16_t entropy_slope;      // Rate of entropy change
6     uint32_t avg_latency_ns;    // P50 latency
7     /* ... */
8 } DictPhysics;
```

The proposed extension gives each VM its own aggregate physics record. It rolls up thermal state across all words (warmest word, average temperature, count of hot words above 0x8000), execution load (cumulative executions, words-per-second rate, recency), memory pressure (dictionary heap usage against capacity), reliability (error count, error rate, recovery count), child VM health, block-device affinity, and lifecycle state (uptime, age, run state, criticality class).

```
1 typedef struct {
2     uint16_t vm_temperature_q8;    // Warmest word temperature
3     uint16_t vm_avg_temperature_q8; // Average across all words
4     uint32_t vm_hot_word_count;    // Words with temp > 0x8000
```

```

5      uint64_t total_word_executions;
6      uint64_t execution_rate_per_sec;
7      float    dictionary_pressure_pct; // used / max
8      uint32_t error_rate_per_sec;
9      uint32_t active_child_vm_count;
10     uint8_t  vm_state;                // RUNNING, PAUSED, DRAINING,
        DEAD
11     uint8_t  vm_criticality;          // system, critical, normal,
        background
12     /* ... */
13 } VMPhysics;

```

The record attaches directly to the `VM` struct. Every physics event maintains it: `physics_metadata_touch()` updates the word, then resolves the current VM and updates the VM-level aggregates, recomputing running averages and rates.

10.1.2 Block Device Physics Metadata

Block storage (1024 blocks of 1024 bytes) is managed by `block_subsystem.c` (logical-to-physical mapping) and `blkio_factory.c` (pluggable RAM, file, and L4Re backends). Today there is no observability beyond capacity tracking. The proposal attaches a `BlockPhysics` record to each block, tracking thermal state (access frequency), access patterns (read and write counts, P50 and P99 latencies), ownership (owning VM and word), content-type hints, and health (CRC32 checksum, dirty and corruption flags, error count). A device-level aggregate summarizes total reads and writes, hottest and coldest block temperatures, and fragmentation percentage.

Block accessors maintain these metrics inline. On read, the subsystem times the I/O, updates the block’s read count and exponential moving-average latency, recomputes temperature from read frequency, and verifies the CRC32 to detect corruption. On write, it does the symmetric work and refreshes the stored checksum.

10.1.3 Mama’s Responsibilities

Mama Forth runs with elevated privileges along three axes:

- **Observe** — all word executions in children, all block-device I/O, all child lifecycle events, all errors and anomalies.
- **Control** — spawn and kill child VMs, allocate blocks to children, enforce governance policies, redirect critical work, scale up and down with load.
- **Coordinate** — share block resources, prevent deadlock, balance load across children, degrade gracefully under stress.

Mama is itself instrumented with a `MamaPhysics` record covering its own supervision load, child-management counts, block-management activity, emergency events (thermal warnings, memory-pressure events, orphan recoveries), and governance metrics (violations detected, enforcement actions, audit events).

10.1.4 The Supervision Loop

Mama runs a periodic control loop. Each cycle observes every child’s VM-level physics, flags thermal warnings, memory pressure above 90%, and error surges, then inspects every block device for hot blocks and excessive fragmentation. It then makes load-balancing decisions, enforces governance compliance, and sleeps for an adaptive interval derived from its own decision latency.

```

1 void mama_supervision_loop(void) {
2     while (mama_vm.running) {
3         for (int i = 0; i < child_vm_count; i++) {
4             VM *child = child_vms[i];
5             VMPhysics p = child->physics;
6             if (p.vm_temperature_q8 > THERMAL_WARNING)
7                 mama_handle_hot_child(child);
8             if (p.dictionary_pressure_pct > 90)
9                 mama_handle_memory_pressure(child);
10            if (p.error_rate_per_sec > ERROR_THRESHOLD)
11                mama_handle_error_surge(child);
12        }
13        mama_balance_load_across_children();
14        mama_check_governance_compliance();
15        sf_sleep_ns(mama_vm.physics.decision_latency_ns * 2);
16    }
17 }

```

Mama spawns a new child when average child temperature exceeds 0xD000 and capacity remains, allocating a block range to the newcomer and rebalancing the work queue. It kills a child that is unrecoverably faulted, idle, or under unrecoverable memory pressure — first recovering the orphan’s blocks, then tearing the VM down. It rebalances block allocation by grouping each child’s hottest blocks together to reduce fragmentation and keep hot data near its consumer.

10.1.5 Three-Tier Feedback Loop

Metrics flow upward. Word execution updates word temperature and latency; the VM aggregates word metrics into VM state; Mama aggregates VM state into system state, which is logged across all three tiers to the analytics heap. Decisions flow downward. A Mama decision (“Child 2 is too hot, spawn Child 3”) creates a VM and allocates blocks; the affected child adjusts its batch groups and sampling rate; word-level optimization boosts priority for hot words; and the block device responds by relocating cold blocks, compacting fragmentation, and prefetching predicted accesses.

10.1.6 Worked Example: Thermal Emergency

A concrete sequence illustrates the loop. A child VM runs normally until a `BLOCK_WRITE` word heats to 0xC000 and propagates that temperature to the VM. Mama’s next cycle detects the warning together with elevated dictionary pressure and an anomalous error rate of 100 errors per second. Mama responds: it spawns a second child for load shedding, reassigns cold work to it, marks the first child’s hot blocks protected, temporarily raises that child’s heap, and logs the emergency to governance. Within a few seconds the original child cools and stabilizes; the system keeps the new child because it now carries useful work, retiring it only once the first child cools below the safe threshold.

10.1.7 Implementation Phases

- **Phase 1 — Foundation.** Word-level physics and execution hooks are complete; VM-level aggregation and block-device physics remain.
- **Phase 2 — Single-VM Control.** Word-level scheduling and memory hints are complete; VM-level health monitoring and observe-only Mama supervision remain.

- **Phase 3 — Multi-VM Orchestration.** Child spawn and kill, block reallocation, cross-child load balancing, and emergency response.
- **Phase 4 — Mama Optimization.** Mama’s own physics tuning, predictive spawning, guided load balancing, and governance enforcement at scale.

Part III

Platform and Hardware

Chapter 11

Platform Abstraction

11.1 The Platform Abstraction Layer

StarForth's platform abstraction layer provides portable timing and clock functionality across POSIX systems (Linux, macOS, BSD) and the L4Re/StarshipOS microkernel, selected by a single Makefile switch. Following the same vtable pattern as the block I/O subsystem, it achieves zero runtime overhead while keeping platform-specific code cleanly separated from the core VM.

11.1.1 Structure

The core VM is platform-agnostic and speaks only to `include/platform_time.h`. Behind that header, compile-time selection routes to one of two backends: `src/platform/time_posix.c` using `clock_gettime`, or `src/platform/time_l4re.c` using the RTC server and KIP clock. Builds choose the path explicitly:

```
1 make                # POSIX (default)
2 make L4RE=1         # L4Re/StarshipOS
3 make MINIMAL=1      # minimal embedded, no platform layer
```

11.1.2 API

All timing flows through `platform_time.h`. The host calls `sf_time_init()` once at startup, then reads `sf_monotonic_ns()` for performance measurement (nanoseconds since boot) and `sf_realtime_ns()` for wall-clock time (nanoseconds since the Unix epoch). `sf_set_realtime_ns()` sets the clock where privileges allow, `sf_format_timestamp()` renders a human-readable string, and `sf_has_rtc()` reports RTC availability. Helper conversions (`sf_seconds_to_ns`, `sf_ns_to_seconds`, `sf_ns_to_ms`, `sf_ns_to_us`) round out the interface.

11.1.3 Backends

The POSIX backend uses `CLOCK_MONOTONIC` for high-precision monotonic time, `CLOCK_REALTIME` for wall-clock time, and `localtime/strftime` for formatting; it always reports an RTC as available. The L4Re backend draws monotonic time directly from the Kernel Info Page clock (`l4re_kip_clock_ns(l4re_kip())`) and real time from the RTC server offset added to that clock. It obtains the "rtc" capability from the L4Re environment, queries the offset over RPC, and degrades gracefully to epoch time when no RTC server is present; setting time updates the offset over RPC and requires a write capability. The selector chooses at compile time on the `__l4__` define:

```
1 void sf_time_init(void) {
2 #ifdef __l4__
```

```
3     sf_time_init_l4re();
4     sf_time_backend = &sf_time_backend_l4re;
5 #else
6     sf_time_backend = &sf_time_backend_posix;
7 #endif
8 }
```

11.1.4 Migration from Direct POSIX Calls

The abstraction replaces scattered platform calls in the core. Where the profiler once called `clock_gettime(CLOCK_MONOTONIC, ...)` directly, it now calls `sf_monotonic_ns()`; where the logger called `time`, `localtime`, and `strftime`, it now calls `sf_realtime_ns()` and `sf_format_timestamp()`. The same source compiles unchanged on both platforms.

11.1.5 Benefits

The layer keeps the core free of `#ifdef` clutter; inline functions compile to direct calls for zero overhead; the vtable gives strong typing; new platforms are easy to add; and the vtable can be swapped for mocks in testing.

11.1.6 StarshipOS Integration and Future Platforms

Copied into `StarshipOS/l4/pkg/starforth`, the build automatically selects the L4Re backend by compiling `time_l4re.c` and `platform_init.c`, linking `librtc`, and defining `__l4__=1`; the loader starts the RTC server first and grants the VM the `rtc` capability. The design anticipates further backends — bare-metal hardware timers, WebAssembly via `performance.now()`, RTOSes such as FreeRTOS and Zephyr, and other microkernels such as seL4. Adding one requires a new `time_<platform>.c`, its backend vtable, a detection branch in `platform_init.c`, and a Makefile flag.

Chapter 12

Linux and POSIX

Chapter 13

L4Re / Fiasco.OC

13.1 L4Re Integration

This guide covers running StarForth on the L4Re runtime environment over the Fiasco.OC microkernel: building it as an L4Re package, managing memory through dataspaces, communicating between VMs by IPC, and the path toward StarshipOS integration.

13.1.1 L4Re Fundamentals

L4Re is a user-level infrastructure layered on the L4 microkernel. It supplies memory management, task and thread management, inter-process communication, device drivers, and runtime libraries. A handful of concepts recur throughout the integration: a *task* is an address space plus its threads; a *dataspace* is a memory object (file-like or anonymous); a *capability* is a reference to a kernel object; an *IPC gate* is a communication endpoint; the *region manager* handles virtual memory; and the *name server* provides service discovery.

13.1.2 Package Structure

StarForth installs into the L4Re source tree as `l4/pkg/starforth`, holding a `Control` metadata file, a top-level `Makefile`, a `server/` directory with the L4Re entry point and VM wrapper, a `lib/` directory wrapping the existing `src/` files, the existing `include/`, and an `examples/` client. The `Control` file declares that the package provides `starforth` and requires `libc`, `libstdc++`, and `l4re-core`.

13.1.3 Build System

The top-level `Makefile` recurses into `lib`, `server`, and `examples`. The library `Makefile` builds `libstarforth` from the full set of VM and word-source files plus an L4Re-specific C++ wrapper, selecting architecture flags from `ARCH` (`-march=x86-64-v2` for amd64, `-march=armv8-a+crc+simd` for arm64) and compiling C with `-std=c99 -O3 -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DL4RE_BUILD=1`. The server `Makefile` builds a static `starforth_server` program requiring `libstarforth`, `l4re_c`, `l4re_c-util`, `libstdc++`, and `libc`.

13.1.4 Memory Management via Dataspaces

Instead of `malloc`, the L4Re wrapper class `L4VM` backs VM memory with a dataspace. Initialization allocates a capability slot (`l4re_util_cap_alloc`), allocates a dataspace of `VM_MEMORY_SIZE` (`l4re_ma_alloc`), and maps it into the address space (`l4re_rm_attach` with `L4RE_RM_F_SEARCH_ADDR | L4RE_RM_F_RW`). The mapped region becomes `vm_.memory`, after which the wrapper performs the usual VM setup — zeroing stack pointers and allotting the `SCR`, `STATE`, and `BASE` cells.

Cleanup detaches the region and frees the capability; the dataspace is reclaimed when its capability is released.

13.1.5 IPC Interface

The server exposes a small opcode protocol: `OP_INTERPRET`, `OP_PUSH`, `OP_POP`, `OP_GET_STATE`, and `OP_RESET`. It is implemented as an `L4::Epiface_t` whose `op_dispatch` reads the opcode from the IPC stream and routes to a handler. `OP_INTERPRET` copies an inbound code buffer, runs `vm_interpret`, and returns the VM error status; `OP_PUSH` and `OP_POP` marshal a single cell; `OP_GET_STATE` returns the stack pointers and error/halt flags; `OP_RESET` tears down and re-initializes the VM. A matching client wraps these calls behind a small C++ class, and registration with the name server makes the server reachable under the `starforth` capability.

13.1.6 Multi-VM Architecture

A typical deployment runs several Forth VMs under a root task (Ned), which acts as name server and VM manager. Distinct VMs serve distinct roles — one hosting Forth tasks, one a request/response server, one an interactive REPL. Ned's `modules.list` wires each VM's capabilities, granting a client the server's IPC channel and giving the server its own fresh channel.

13.1.7 Kernel-Context Forth

Running Forth inside the kernel is supported for narrow uses — runtime tunables, device-driver hotpatching, and interactive kernel debugging — and demands extreme care. Kernel-context VMs have no `malloc/free` (they use a static memory pool), no syscalls, limited stack, and must be interrupt-safe and non-blocking. The kernel module registers only a minimal, side-effect-free word set (arithmetic, logical, stack) and explicitly omits I/O and blocking words. An entry point executes trusted code through `vm_interpret`, and a polling-serial REPL supports live debugging.

13.1.8 StarshipOS, Building, and Operations

On StarshipOS the natural integration points are a boot service that starts the StarForth server, system configuration expressed in Forth, runtime hot-patching, and an interactive debugging REPL; a boot script such as `/boot/init.fth` can register services and set kernel parameters during startup. Packages build per-architecture with `make ARCH=amd64` or `ARCH=arm64` (with `CROSS_COMPILE=aarch64-linux-gnu-` for cross-builds), and boot images assemble through a `modules.list` and run under QEMU. Performance work centers on huge-page-aligned dataspace, shared memory for large transfers in place of inline IPC, and CPU affinity for cache locality. Debugging spans GDB over QEMU's stub, the Fiasco JDB kernel debugger, and L4Re's `Dbg` logging. Security rests on L4Re's capability model: VMs receive only the capabilities they need and run sandboxed under Ned configuration, deliberately withholding scheduler and allocator access from untrusted code.

13.2 Dictionary Allocation on L4Re

StarForth currently allocates dictionary entries from the host C library. The `vm_create_word()` routine calls `malloc()` for each `DictEntry`, and `FORGET` releases them with `free()`. This is correct and well-tested on hosted systems, but it ties the dictionary to a full libc heap — an awkward dependency in a microkernel context.

13.2.1 Why It Matters for L4Re

The libc-heap approach carries four liabilities under L4Re: it requires a complete `malloc` implementation in the microkernel environment; heap fragmentation makes allocation behavior non-deterministic; dictionary entries live outside the VM's own memory model in a separate address space; and the VM cannot be introspected or snapshotted as a single contiguous region.

A VM-based allocator answers each point. It removes the external dependency, a simple bump allocator behaves deterministically, everything lives inside one 5 MB VM memory region, the entire VM state can be serialized at once, and the model aligns naturally with L4Re datas-paces.

13.2.2 Proposed Approach: Conditional Compilation

A compile-time flag selects between the two strategies, leaving the hosted path untouched:

```

1  #ifdef STARFORTH_VM_DICT_ALLOC
2      DictEntry *entry = dict_alloc_from_vm(vm, total);    /* L4Re/
                        embedded */
3  #else
4      DictEntry *entry = (DictEntry *) malloc(total);      /* hosted */
5  #endif

```

In VM-based mode the 5 MB region is laid out with a 256 KB dictionary-header area at the base (growing upward as entries are allocated), then user data and compiled code (growing upward from `HERE`), with block storage filling the remainder. The allocator itself is a cell-aligned bump pointer:

```

1  static DictEntry *dict_alloc_from_vm(VM *vm, size_t total_bytes) {
2      size_t align = sizeof(cell_t);
3      total_bytes = (total_bytes + align - 1) & ~(align - 1);
4      if (vm->dict_headers_used + total_bytes > DICT_HEADERS_MAX) {
5          vm->error = 1;
6          return NULL;
7      }
8      DictEntry *entry = (DictEntry *) (vm->memory + vm->
          dict_headers_used);
9      vm->dict_headers_used += total_bytes;
10     return entry;
11 }

```

Because allocation is a bump pointer, `FORGET` becomes a rewind rather than a loop of frees: it sets `dict_headers_used` back to the target entry's offset. VM initialization starts `dict_headers_used` at zero and offsets `HERE` to `DICT_HEADERS_MAX`. The fast-lookup cache is unaffected — its bucket arrays may still use `malloc` while the entries themselves live in VM memory.

13.2.3 Scope of Change

The switch touches only three functions plus one new field. The new field is `size_t dict_headers_used` in the VM struct. The functions are `vm_create_word()` (use the VM allocator), the `FORGET` implementation (rewind instead of freeing), and `vm_init()` (initialize the offset and `HERE`). Estimated effort is roughly six hours: four to implement and two to test.

13.2.4 Current Status

The present decision is to retain `malloc()` and document the abstraction point clearly. The hosted implementation is stable and well-tested, the two approaches show no performance differ-

ence, and the abstraction is simple enough that the L4Re port can flip the switch when needed. The action items for that port are to define `STARFORTH_VM_DICT_ALLOC` in the L4Re build, implement `dict_alloc_from_vm()`, convert `FORGET` to a rewind, run the full test suite, and verify no leaks in the microkernel environment.

13.3 Block I/O Porting Endpoints for L4Re

The `blkio` subsystem maps cleanly onto L4Re because its northbound API never changes. Porting to the microkernel means supplying new backends — dataspace-based or IPC-based — behind the same six-function interface; the VM and Forth block words keep calling exactly what they call today.

13.3.1 The Stable Northbound API

Every caller goes through this fixed surface, which constitutes the subsystem ABI:

```

1  int  blkio_open (blkio_dev_t *dev, const blkio_vtable_t *vt, const
      blkio_params_t *p);
2  int  blkio_close(blkio_dev_t *dev);
3  int  blkio_read (blkio_dev_t *dev, uint32_t fblock, void *dst);
4  int  blkio_write(blkio_dev_t *dev, uint32_t fblock, const void *src)
      ;
5  int  blkio_flush(blkio_dev_t *dev);
6  int  blkio_info (blkio_dev_t *dev, blkio_info_t *out);

```

13.3.2 In-Process Dataspace Backend

When the VM can attach a dataspace directly, this is the fastest path — no IPC involved. The backend exposes `blkio_l4ds_state_size()`, `blkio_l4ds_init_state(...)`, and `blkio_l4ds_vtable()`, and holds a small state record carrying the dataspace capability, the attached virtual address, the mapped length, the block size, the block count, and a read-only flag. The operations are direct: `open` attaches the dataspace with `l4re_rm_attach()` and derives the block count; `read/write` are `memcpy` against the mapped region; `flush` is a no-op (or a dataspace operation if required); and `close` detaches with `l4re_rm_detach()`.

13.3.3 Out-of-Process Block Server Backend

When storage is owned by another task — an AHCI, NVMe, or eMMC server — the client-side backend exposes `blkio_l4svc_state_size()`, `blkio_l4svc_init_state(...)`, and `blkio_l4svc_vtable()`, talking to the server over a small opcode protocol:

```

1  enum bsvc_op {
2      BSVC_INFO   = 1,
3      BSVC_READ   = 2,
4      BSVC_WRITE  = 3,
5      BSVC_FLUSH  = 4,
6      BSVC_PING   = 5
7  };

```

Two transports are available: a shared dataspace mapped by both server and client (preferred), or inline IPC carrying the block payload in the message (simpler but less efficient). The operations map onto the protocol directly — `read` sends the opcode and block number and the server fills the shared buffer, `write` copies into the shared buffer before sending, and `info` queries block size, count, and flags.

13.3.4 Factory Helpers and CLI Semantics

Backend selection can be hidden behind factory overloads — `blkio_factory_open_l4ds(...)`, `blkio_factory_open_file(...)`, and `blkio_factory_open_l4svc(...)` — so `main.c` chooses a backend from its arguments and environment. On L4Re the CLI follows the same logic: `-disk-img=<path>` resolves either to a file server returning a dataspace (the `l4ds` backend) or to a block server (the `l4svc` backend); `-ram-disk=<MB>` allocates a RAM dataspace and uses the `l4ds` backend; and with no disk image the system falls back to a RAM disk from the allocator.

13.3.5 Composite Backends

The intended layout dispatches by block number: blocks 0–1023 go to a RAM backend and blocks 1024 and above go to a disk backend, both wrapped in a composite vtable that routes on `fblock`. A later enhancement adds an outer wrapper that packs three 1 KiB blocks plus 1 KiB of metadata into 4 KiB pages for device alignment — still presenting the same five operations.

13.3.6 Summary

The L4Re port introduces two new backends (the in-process dataspace backend and the IPC client/server backend) and, optionally, two factory helpers. Nothing else moves: the VM continues to call `blkio_read`, `blkio_write`, and the rest. Backends are swapped, not call sites.

Chapter 14

Raspberry Pi 4 / ARM64

14.1 Building StarForth on Raspberry Pi 4

This guide covers building and tuning StarForth for the Raspberry Pi 4 with its ARM64 assembly optimizations. The board pairs a Broadcom BCM2711 (quad-core Cortex-A72 at 1.5 GHz, ARMv8-A) with 32 KB of L1 instruction and data cache per core, 1 MB of shared L2, and LPDDR4-3200 memory; NEON and CRC32 are both present.

14.1.1 Building

The Pi 4 can be built natively or cross-compiled. A native build installs `build-essential`, clones the repository, and compiles with the Cortex-A72 tuning flags:

```
1 make CFLAGS="-std=c99 -O3 -march=armv8-a+crc+simd -mtune=cortex-a72 -\
2 -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -Iinclude -\
3 -Isrc/word_source -Isrc/test_runner/include"
```

Cross-compiling from x86_64 Linux installs `gcc-aarch64-linux-gnu`, builds with `CC=aarch64-linux-gnu-gcc` and `LDFLAGS="-static"`, and copies the static binary to the board over `scp`. The repository provides convenience targets: `rpi4` for a native build, `rpi4-cross` for a static cross-build, and `rpi4-perf` for the maximum-performance build with direct threading and LTO.

14.1.2 Architecture Detection and Code Integration

A detection header (`include/arch_detect.h`) selects the right optimization headers and primitive macros at compile time — `x86_64`, `AArch64`, or a portable fallback with a warning. Stack management, arithmetic words, and dictionary lookup each keep an assembly path guarded by `USE_ASM_OPT` alongside a C path, so the architecture-agnostic `vm_push_opt/vm_pop_opt` macros resolve to the best available implementation.

14.1.3 Performance Tuning

Three techniques dominate on the Cortex-A72. Aligning the VM struct to the 64-byte cache line and grouping hot data (stacks and pointers) away from cold data reduces false sharing. Prefetching the next dictionary entry while comparing the current one hides traversal latency. And NEON block copy moves 64 bytes per iteration for bulk memory operations, falling back to scalar copy for the remainder.

14.1.4 Benchmarking

Reliable measurement requires pinning the CPU to performance mode and watching temperature. The governor is set with:

```
1 echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/
  scaling_governor
2 watch -n 1 vcgencmd measure_temp
3 vcgencmd get_throttled          # 0x0 means no throttling occurred
```

A benchmark script builds an -O2 baseline and the optimized `rpi4-perf` binary, runs a million iterations each of stack, arithmetic, and logic loops, and reports the timing difference along with final temperature and throttling status. The `perf` tool supplies cache-reference, cache-miss, branch, and branch-miss counters for deeper analysis.

14.1.5 Thermal and Power

Sustained workloads need cooling. A passive aluminum or copper heatsink covering the entire CPU with a 1 mm thermal pad is the minimum; a 5 V 30 mm fan, optionally GPIO-controlled, handles continuous load. Power draw runs 2–3 W idle, 4–5 W under light load, and 7–8 W with all cores busy, plus 2–3 W for peripherals — the official 5 V/3 A supply is recommended. Overclocking through `/boot/config.txt` is possible but requires adequate cooling and may void the warranty.

14.1.6 Storage and Deployment

For the SD card, choose UHS-I or better with an A2 application-performance rating; brands with strong random I/O include Samsung EVO Plus, SanDisk Extreme, and Kingston Canvas React. For production, a systemd service unit runs StarForth on boot with automatic restart on failure; boot time can be trimmed by disabling unused features and the splash screen in `/boot/config.txt`; and standard hardening (system updates, a UFW firewall, and key-only SSH) rounds out a deployment.

14.1.7 Troubleshooting

- *Illegal instruction at runtime* — the build exceeded the host’s feature set; rebuild with the conservative `-march=armv8-a`.
- *Cross-compiled binary will not run* — confirm it is statically linked (`file build/starforth`); rebuild with `-static` otherwise.
- *Slower than expected* — verify the governor is `performance` rather than `ondemand` and check `vcgencmd get_throttled` for thermal throttling.
- *Out of memory* — reduce `VM_MEMORY_SIZE` or add swap.

14.2 ARM64 Assembly Optimizations

StarForth’s ARM64 optimizations target the AArch64 instruction set, validated primarily on the Cortex-A72 of the Raspberry Pi 4. The work spans four headers: `include/vm_asm_opt_arm64.h` (core optimizations), `include/vm_inner_interp_arm64.h` (direct-threaded interpreter), and `include/arch_detect.h` (automatic architecture detection), with a full build guide in the Raspberry Pi chapter.

14.2.1 Architectural Advantages

ARM64 offers several structural wins over x86_64 for a stack machine. With 31 general-purpose registers against 16, the implementation keeps the VM pointer, instruction pointer, and both stack pointers in registers and still has room to cache the top of stack (TOS) in `x23` — eliminating a memory access on every operation — with `x24–x28` held in reserve. Most ARM64 instructions carry conditional variants (`cse1`, `cneg`, `cinc`) where x86_64 offers only `CMOV`, removing branches and the pipeline stalls they cause. Load and store with auto-increment fuse a memory access and a pointer bump into a single instruction, shrinking code and easing i-cache pressure. NEON SIMD compares sixteen bytes at once, accelerating string comparison and bulk memory work.

Feature	x86_64	ARM64 (Cortex-A72)
General-purpose registers	16	31
TOS caching	Limited	Excellent (<code>x23</code>)
Conditional execution	<code>CMOV</code> only	Most instructions
Load/store	Complex modes	Post-increment
SIMD width	256-bit (AVX2)	128-bit (NEON)
Power envelope	15–25 W TDP	7–8 W

Table 14.1: Architectural comparison relevant to the StarForth inner loop.

14.2.2 Expected Gains

Optimization	x86_64 speedup	ARM64 speedup
Stack operations	2–3×	2.5–4×
Inner interpreter	3–5×	4–6×
Arithmetic	1.5–2×	1.8–2.5×
Dictionary lookup	2–3×	2–3×
String operations	2–3×	2–3× (NEON)

Table 14.2: Projected speedups by optimization class.

14.2.3 TOS Caching in Practice

Because the top of stack lives in `x23` and the data-stack pointer `x21` is a true pointer rather than an index, a push collapses to a single auto-incrementing store:

```

1 ; TOS already in x23, no load needed
2 str      x23, [x21, #8]!      ; store TOS, advance DSP in one
   instruction

```

Dictionary traversal benefits from prefetching the next entry while comparing the current one. Each entry carries roughly 100–200 ns of latency; a prefetch hides 50–80 ns of it, yielding a 30–50% speedup on cold searches.

14.2.4 Raspberry Pi 4 Target

The Cortex-A72 runs four out-of-order cores at 1.5 GHz with a 4096-entry branch-prediction buffer. Each core has 32 KB of L1 instruction and 32 KB of L1 data cache; 1 MB of L2 is shared. Memory bandwidth on LPDDR4-3200 is roughly 12 GB/s theoretical and 8–10 GB/s measured. Practical guidance follows from the cache hierarchy: keep hot code under 32 KB to fit L1I,

align critical loops to cache lines, prefetch predictable access patterns, structure data on 64-byte boundaries, and minimize memory traffic by keeping working data in registers.

14.2.5 Build Configuration

Architecture detection selects flags from `uname -m`: `-march=native` on x86_64, and `-march=armv8-a+crc+simd` `-mtune=cortex-a72` on AArch64, each with a matching `ARCH_*` define. The optimized profile adds `-O3 -DUSE_ASM_OPT=1`; the performance profile adds `-DUSE_DIRECT_THREADING=1 -flto`. Cross-compilation uses `aarch64-linux-gnu-gcc` with static linking.

14.2.6 Benchmark Results

Measured on a Raspberry Pi 4 Model B (4 GB) running 64-bit Raspberry Pi OS, kernel 6.1.21-v8+, GCC 12.2.0.

Implementation	Time	Speedup
C baseline (-O2)	285 ms	1.0×
C optimized (-O3)	198 ms	1.4×
ARM64 ASM	68 ms	4.2×
ARM64 + direct threading	42 ms	6.8×

Table 14.3: One million stack operations.

For recursive Fibonacci(30), direct threading eliminated roughly 1.7 billion branches (an 83% reduction), cut instruction count by 60%, and reduced cache misses by 75%, bringing the C baseline of 1250 ms down to 245 ms. Dictionary lookup over 1000 words across 100k searches fell from 89 ms (45k cache misses) to 41 ms (25k misses) with combined prefetch and NEON `strcmp`.

14.2.7 Energy Efficiency

ARM64 optimizations raise instantaneous power slightly through higher utilization but complete work far faster, lowering energy per operation. One million stack operations cost roughly 1.2 J on the C baseline (285 ms $\times$ 4.2 W) versus 0.19 J optimized (42 ms $\times$ 4.5 W) — a 6.3× improvement in energy efficiency. Sustained workloads warrant at least a passive heatsink; without cooling the Cortex-A72 throttles to 1.2 GHz near 80 °C.

14.2.8 Known Limitations

- The NEON string compare assumes alignment and may fault on unaligned input; an alignment check or unaligned loads would resolve it.
- `vm_mul_double` produces correct low and signed-high 64-bit halves, but unsigned 128-bit division is unimplemented; `*/MOD` falls back to software division.
- Cache-line zeroing via `dc zva` requires an aligned address and may be disabled by the hypervisor or kernel.
- The assembly is validated on Cortex-A72; other ARM64 cores (A53, A76, Apple M-series) may need retuning and should be benchmarked on the target.

14.2.9 Portability and Future Work

The same ARM64 code runs on Apple M1/M2 (wider execution and a much larger L2, expected $2\text{--}3\times$ over the Pi 4, built with `-mcpu=apple-m1`), AWS Graviton (`-mcpu=neoverse-n1`), and Android devices via the NDK toolchain. Future directions include NEON parallel stack operations, the Scalable Vector Extension on ARMv9, and the ARMv8.3+/8.5 security features Pointer Authentication and Branch Target Identification.

Chapter 15

Performance Profiling

15.1 The Built-In Word Profiler

StarForth ships a lightweight word-execution profiler that tracks how often each word runs and turns that data into optimization guidance. Its purpose is to surface hot paths — frequently executed words — that are candidates for inline assembly. At its basic level the profiler adds well under 1% overhead, making it suitable for always-on use.

15.1.1 Quick Start

Profiling is enabled with two flags: `-profile` sets the detail level and `-profile-report` prints the report on exit.

```
1 ./build/starforth --profile 1 --profile-report < script.fth
```

The report lists global statistics and the most frequently called words by execution count — typically dominated by `LIT`, `EXIT`, `(LOOP)`, `I`, and the core stack and arithmetic primitives.

15.1.2 Profiling Levels

Level	Overhead	Adds
0 — <code>PROFILE_DISABLED</code>	0%	Nothing (production default)
1 — <code>PROFILE_BASIC</code>	<1%	Frequency & lookup counts
2 — <code>PROFILE_DETAILED</code>	5–10%	Per-word timing (ns)
3 — <code>PROFILE_VERBOSE</code>	15–20%	Stack & memory access tracking

Table 15.1: Profiling levels and their cost.

Basic profiling tracks word frequency and dictionary lookups at negligible cost. Detailed profiling adds nanosecond-precision execution timing with average, minimum, and maximum per word. Verbose profiling further records stack operation counts and memory read/write volumes.

15.1.3 Reading the Report

High call counts mark hot paths. The detailed view reports total time per word in microseconds alongside average and maximum nanoseconds, exposing words that are individually slow as well as those that are merely frequent. A typical workload concentrates 80–95% of all executions in its top ten words, so a small set of targeted optimizations affects nearly all runtime.

15.1.4 Hot-Word Analysis

The `profiler_print_hotspots()` routine ranks words by share of total executions and attaches a recommendation to each.

Share of total	Recommendation
$\geq 5.0\%$	High priority — inline-assembly candidate
$\geq 2.0\%$	Consider assembly optimization
$\geq 1.0\%$	Monitor for optimization
$\geq 0.5\%$	Defer to profile-guided optimization
$< 0.5\%$	Low priority

Table 15.2: Optimization priority by execution share.

15.1.5 Development Workflow

The intended cycle is: profile a representative workload at the basic level, identify hot words, then optimize. Words above 5% of executions warrant a hand-written assembly path under a `USE_ASM_OPT` guard; words in the 0.5–5% band are better left to profile-guided optimization, where the compiler inlines and arranges them from the same data. Gains are confirmed by benchmarking before and after with `-benchmark`.

Best practice is to profile real programs rather than toy examples, run enough iterations for statistical significance (1000+ word executions), and start at the basic level. Profiling should avoid `LOG_DEBUG` (logging overhead skews results), focus on normal rather than error paths, and treat a top-ten coverage below 80% as a sign the workload is unrepresentative.

15.1.6 Implementation

The profiler instruments execution in two places: the outer interpreter (`vm_interpret_word()`), which sees words run directly from the REPL or scripts, and the inner interpreter (`execute_colon_word()`), which sees words run from compiled threaded code. Per-word statistics are held in a compact record:

```

1 typedef struct {
2     const DictEntry *entry;      // dictionary entry
3     uint64_t call_count;        // executions
4     uint64_t total_time_ns;     // DETAILED+
5     uint64_t min_time_ns;      // DETAILED+
6     uint64_t max_time_ns;      // DETAILED+
7 } WordStats;

```

Frequency tracking is a single counter increment with $O(1)$ lookup for known entries and $O(n)$ only on a word's first call, with capacity for 256 unique words (expandable; primitive words are always tracked). It performs no timing calls, which is what keeps the basic level effectively free.

15.1.7 C API

```

1 int profiler_init(ProfileLevel level);
2 void profiler_shutdown(void);
3 void profiler_word_count(const DictEntry *entry); // lightweight
4 void profiler_word_enter(const DictEntry *entry); // timing
5 void profiler_word_exit(const DictEntry *entry);
6 void profiler_generate_report(void);

```

```

7 void profiler_print_hotspots(void);
8 void profiler_reset(void);

```

15.1.8 Integration with External Tools

The built-in profiler complements rather than replaces system profilers. It reports hot *Forth* words; **perf** reports hot *C* functions; Valgrind/Callgrind supplies exact instruction counts; and **gprof** provides function-level timing. The recommended combination runs the StarForth profiler at the basic level continuously and reaches for the detailed level or an external tool only for targeted investigation. Typical failures are simple: no data usually means one of the two flags is missing, all-zero counts mean no Forth code executed before **BYE**, excessive overhead means a level above basic is in use, and missing words mean the 256-entry capacity was reached.

15.2 x86_64 Assembly Optimizations

StarForth's x86_64 assembly optimizations serve both standard Linux environments and L4Re microkernel deployments. They live in two headers: `include/vm_asm_opt.h` (basic stack and arithmetic primitives) and `include/vm_inner_interp_asm.h` (the direct-threaded inner interpreter).

15.2.1 Performance Impact

Optimization	Speedup	Use case
Stack operations	2–3×	Every word execution
Inner interpreter	3–5×	Colon definitions
Arithmetic operations	1.5–2×	Math-heavy code
Dictionary lookup	2–3×	Compilation/interpretation
String operations	1.5–2×	Text processing

Table 15.3: Measured impact by optimization class.

15.2.2 Enabling the Optimizations

The optimizations are gated by preprocessor flags. A standard build enables both stack/arithmetic primitives and direct threading; an L4Re build typically targets a conservative microarchitecture level:

```

1 # Standard
2 CFLAGS += -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -O3 -march=native
3
4 # L4Re/StarshipOS
5 CFLAGS += -DUSE_ASM_OPT=1 -march=x86-64-v2 -O3

```

Each optimized word keeps both an assembly and a C path behind a `USE_ASM_OPT` guard, so a build with the flag disabled falls back to the portable implementation. Stack operations, arithmetic words, and dictionary lookup all follow this pattern — the C path provides correctness and the assembly path provides speed.

15.2.3 Direct-Threaded Inner Interpreter

This is the highest-impact and most intricate optimization. A naive C interpreter dispatches each word through a function call:

```

1 void execute_colon_word(VM *vm) {
2     cell_t *ip = vm->ip;
3     while (*ip) {
4         DictEntry *word = (DictEntry*)*ip++;
5         word->func(vm);           // call/return overhead on every
                                   word
6         if (vm->exit_colon) break;
7     }
8 }

```

The call and return per word are expensive, branch prediction suffers, and registers spill. The direct-threaded path instead loads the instruction pointer and stacks into registers once, lets each primitive do its work and jump straight to the next, and saves registers only on exit. Primitives are written as thin macro bodies followed by `NEXT_ASM()`:

```

1 void forth_dup_fast(void) { PRIM_DUP(); NEXT_ASM(); }
2 void forth_plus_fast(void) { PRIM_PLUS(); NEXT_ASM(); }
3 void forth_fetch_fast(void) { PRIM_FETCH(); NEXT_ASM(); }

```

15.2.4 Benchmarking

A small Forth harness exercises the hot paths — `DUP/DROP`, arithmetic, and multi-item stack churn — over a million iterations each. The recommended methodology builds three binaries (baseline `-O2`, assembly `-O3`, and assembly plus direct threading), times each, and uses `perf record/perf report` and `perf stat -r 10` for detailed and repeated measurement.

15.2.5 L4Re / StarshipOS Integration

For the microkernel, the assembly optimizations build with `-march=x86-64-v2` and, in kernel context, `-fno-stack-protector` and `-mno-red-zone`, linking against `l4re-util` and `l4sys`. VM memory is allocated from an L4Re dataspace (`l4re_ma_alloc` plus `l4re_rm_attach`) rather than the host heap, and inter-VM communication packs stack data into IPC message registers via `l4_ipc_send`.

15.2.6 Debugging and Correctness

Assembly can be disabled wholesale with `-DUSE_ASM_OPT=0 -O0 -g` for debugging; under GDB the relevant state lives in `r12-r15`. Correctness is verified by running both the C and assembly paths on identical inputs and asserting equal results and stack pointers. The assembly sets `vm->error` on overflow or underflow but does not log, for performance; debug builds add logging through a macro that compiles away in release.

15.2.7 Platform Compatibility

Platform	Stack ops	Arithmetic	Direct threading
Linux x86_64	Yes	Yes	Yes
L4Re x86_64	Yes	Yes	Yes
StarshipOS	Yes	Yes	Yes (kernel & user)
ARM64	—	—	— (separate path)
RISC-V	—	—	— (future work)

Table 15.4: x86_64 assembly optimization support by platform.

15.2.8 Safety and Tuning

Guard pages around the data and return stacks (`mprotect` with `PROT_NONE`) provide hardware backstops beyond the in-line overflow checks. Further gains come from profile-guided optimization, cache-line alignment of the VM struct, dictionary-entry prefetching, and huge pages for VM memory. Common failures map to clear fixes: an illegal-instruction fault usually means `-march=native` exceeded the host's feature set (drop to `-march=x86-64-v2`); crashes in assembly typically trace to stack misalignment (the ABI requires 16-byte alignment) or incorrect clobber lists; and a lack of improvement calls for `perf` to locate the true bottleneck and a check that the optimizations are actually enabled.

15.3 Profile-Guided Optimization

Profile-guided optimization (PGO) feeds runtime profiling data back into the compiler to direct code generation. StarForth's PGO pipeline exercises every major code path, producing a binary tuned to real-world execution patterns. For typical workloads PGO yields a **5–15% improvement** over a standard `-O3` build by predicting hot paths correctly, packing hot code for instruction-cache locality, inlining frequently called functions more aggressively, and laying out code to match common execution order.

15.3.1 Quick Start

A complete PGO build runs from a single target:

```
1 make pgo
```

The target executes a six-stage pipeline: clean the environment, build with instrumentation (`-fprofile-generate`), run the comprehensive profiling workload, collect the resulting `.gcda` files, rebuild with the profile data (`-fprofile-use`), and clean up temporary artifacts. Build time is roughly two to three minutes depending on hardware.

15.3.2 PGO Targets

- `make pgo` — maximum performance for typical workloads. Runs the full profiling workload and produces an optimized binary with assembly optimizations and direct threading enabled, then removes profile data automatically.
- `make pgo-perf` — everything in the standard build plus `perf` capture during workload execution, retaining frame pointers for accurate stack traces. Produces `pgo-perf.data` for call-graph and hotspot analysis. Requires `sudo` and `linux-tools-generic`.
- `make pgo-valgrind` — runs Callgrind during the benchmark workload, collecting instruction counts, cache misses, and branch mispredictions, and builds an optimized binary retaining debug symbols. Produces `pgo-callgrind.out`. Callgrind imposes a 10–100× slowdown, so the workload is limited to 100 benchmark iterations.

15.3.3 Benchmark Comparison

The `make bench-compare` target builds both a regular and a PGO binary, runs identical benchmarks, and reports the timing difference. A representative run shows the regular build at 0.45s elapsed and the PGO build at 0.38s — roughly a 1.18× speedup.

15.3.4 The Profiling Workload

The workload script (`scripts/pgo-workload.sh`) drives seven major code paths: the full unit test suite (over 400 cases across all word categories), stress tests (deep call stacks, stack exhaustion, large definitions), integration tests (complete Forth programs and real-world usage), benchmarks (5000 iterations of hot-path operations), an interactive REPL workload, block I/O operations, and lightweight word-frequency profiling for hot-word identification. The workload can be run independently against any binary:

```
1 ./scripts/pgo-workload.sh ./build/starforth
```

15.3.5 Profile Data

GCC emits two file types. The `.gcno` coverage notes are written at compile time and removed before the optimization build; the `.gcda` coverage data is written at runtime and consumed by `-fprofile-use`. Data lands in the directory where the instrumented binary runs; the build system searches `./`, `src/`, `src/*/`, and `build/`. A healthy run produces 100 or more `.gcda` files, one per source file that executed; substantially fewer indicates unexercised code paths.

15.3.6 Compiler Flags

The instrumentation stage uses moderate optimization to keep the profiling build fast:

```
1 CFLAGS="-O2 -DUSE_ASM_OPT=1 -fprofile-generate"
2 LDFLAGS="-fprofile-generate -lgcov"
```

The optimization stage applies maximum optimization, direct threading, and graceful handling of inconsistent profile data:

```
1 CFLAGS="-O3 -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 \
2 -fprofile-use -fprofile-correction -Wno-error=coverage -
  mismatch"
3 LDFLAGS="-fprofile-use"
```

For `perf` analysis, `-fno-omit-frame-pointer` preserves the frame-pointer register at a 2–3% performance cost in exchange for accurate call graphs.

15.3.7 Troubleshooting

- *No profile data found* — the workload failed to exercise the relevant code, the `.gcda` files are missing, or the source changed between stages. Expand the workload and rebuild with `make pgo`, which cleans first.
- *Coverage mismatch* — source was modified between instrumentation and optimization. Run `make pgo` from scratch.
- *Permission denied writing .gcda* — the working directory is not writable; restore write permissions and rebuild.
- *PGO slower than the regular build* — the profile does not match real usage, or the workload was unrealistic. Customize the workload and confirm with `make bench-compare`.

15.3.8 Best Practices

Profile representative workloads that cover the bulk of real usage rather than trivial operations or rarely executed error paths. Re-run PGO after major code changes (more than roughly 10% of the codebase) and always verify gains with `make bench-compare`. PGO composes well with

`-march=native`, link-time optimization, direct threading, and hand-written assembly — `make pgo` enables all of these automatically.

15.3.9 How PGO Works

During instrumentation the compiler inserts counters at control-flow edges; the runtime increments them and writes `.gcda` files on exit. During optimization the compiler reads the profile, separates hot from cold paths, and makes informed decisions: inlining hot functions, packing hot code for I-cache locality, arranging branches for correct prediction, devirtualizing calls where the profile fixes the type, and unrolling hot loops. Hot code is laid out sequentially while cold code is outlined to a separate section.

Heuristic	Default	With PGO
Inline threshold	600 units	Adjusted per call site
Loop unroll factor	4	Up to 8 for hot loops
Branch prediction	Static (50/50)	Dynamic (from profile)
Function outlining	Disabled	Cold code outlined
Register allocation	Balanced	Favors hot paths

Table 15.5: GCC optimization heuristics with and without profile data.

15.3.10 Build Target Comparison

Target	Optimization	Speed	Build Time	Use Case
<code>debug</code>	<code>-O0 -g</code>	1.0×	30 s	Development
<code>all</code>	<code>-O2</code>	3.5×	45 s	Default, balanced
<code>fast</code>	<code>-O3 + ASM</code>	5.2×	60 s	Production, no LTO
<code>fastest</code>	<code>-O3 + ASM + DT + LTO</code>	6.8×	90 s	Maximum
<code>pgo</code>	<code>fastest + PGO</code>	7.5–8.0×	3 m	Absolute maximum

Table 15.6: Relative performance and build cost across targets (ASM: assembly optimizations; DT: direct threading; LTO: link-time optimization).

15.3.11 Platform Notes

On `x86_64` all PGO features are available, with best results under `-march=native`; `perf` hardware counters and Valgrind are fully supported. On ARM64 (Raspberry Pi 4, Apple Silicon) PGO is fully supported — use `-march=armv8-a+crc+simd -mtune=cortex-a72` on the Pi 4; `perf` support varies by kernel and Valgrind support is limited on some platforms. PGO requires native execution to profile, so cross-compilation must run the instrumented binary on target hardware, transfer the `.gcda` files back, and cross-compile the optimized binary with that data.

Part IV

Specifications

Chapter 16

HOLA Analytics Protocol

16.1 The HOLA Shared-Memory Protocol (Phase 1)

Phase 1 establishes the Host Observation & Logistics Adapter (HOLA) as the stable contract between the StarForth VM and external analyzers. The runtime exposes a fixed-size analytics heap — 10 MiB by default — implemented in `physics_runtime.c`. This section specifies the shared-memory schema and command flow that Phase 1 tooling expects.

16.1.1 Memory Layout

The analytics heap divides into four regions. The header carries single-producer/single-consumer metadata identified by `HOLA_SHARED_MAGIC`; the event ring is a lock-free buffer of physics events; the summary region holds aggregated statistics and Bayesian posteriors; and the scratch region is reserved for governance-approved features.

Region	Offset	Size	Notes
Header	0	<code>sizeof(header_t)</code>	Producer/consumer metadata
Event ring	<code>ring_offset</code>	<code>ring_bytes</code>	Lock-free event ring
Summary	<code>summary_offset</code>	<code>summary_bytes</code>	Stats & posteriors
Scratch	<code>scratch_offset</code>	<code>scratch_bytes</code>	Reserved

Table 16.1: HOLA analytics heap regions.

The header mirrors the C structure in `include/physics_runtime.h` and must remain packed and aligned exactly as defined. All multi-byte fields use the VM's native endianness. Consumers must validate `magic`, `version_major`, and `version_minor` before touching the heap.

16.1.2 Event Records

Each ring entry begins with a fixed header:

```
1 struct physics_analytics_event_header {
2     uint32_t channel;           // logical stream id
3     uint16_t payload_bytes;     // raw payload size
4     uint16_t reserved;         // alignment padding (zero)
5     uint64_t timestamp_ns;     // producer timestamp (sf_monotonic_ns)
6 };
```

Payloads are padded to eight bytes. The producer updates `produce_seq` atomically after writing each record; consumers advance `read_offset` and bump `consume_seq` once an event is fully processed. Overflow is signalled by incrementing `dropped_events` while leaving the ring untouched, and consumers must treat the next readable event as the resynchronization point.

16.1.3 Channels

Channel	Purpose
0x00000001	Word execution samples (entropy, latency, dictionary id)
0x00000002	Host snapshot deltas (<code>physics_host_snapshot_t</code>)
0x0000FF00–0x0000FFFF	Governance-reserved control/alert events

Table 16.2: Recommended HOLA channels.

Channel 0x00000002 carries a binary `physics_host_snapshot_t` in native endianness. Phase 1 adds Pressure Stall Information averages (`avg10/60/300` scaled by 1000), `/proc/stat` totals (`cpu_total_jiffies`, `cpu_idle_jiffies`) for consumer-side delta calculation, cgroup v2 readings (`cgroup_cpu_usage_us` from `cpu.stat`, `cgroup_memory_current_bytes` from `memory.current`, each zero when unavailable), and a `flags` bitmask (`PHYSICS_HOST_FLAG_*`) indicating which groups were populated. Consumers should honor the flag bits to distinguish kernels that expose PSI, cgroup v2, or `/proc/stat`.

16.1.4 Command Protocol

Commands travel through the summary region; its first 256 bytes form a simple request/response mailbox:

```

1 struct hola_command {
2     uint32_t opcode;           // see table
3     uint32_t arg0;             // optional argument / status
4     uint64_t arg1;             // optional payload offset from heap base
5 };

```

Opcode	Direction	Semantics
HOLA_NOP	Either	No-op; producer clears when idle
HOLA_REQUEST_SNAPSHOT	Consumer → VM	Publish snapshot on channel 0x2
HOLA_RESET_RING	Consumer → VM	VM zeroes cursors when safe
HOLA_STATUS_OK	VM → Consumer	Command completed
HOLA_STATUS_BUSY	VM → Consumer	Deferred; retry next tick
HOLA_STATUS_UNSUPPORTED	VM → Consumer	Opcode not understood

Table 16.3: HOLA command opcodes.

The VM polls the mailbox at the end of each observation-window hop and writes responses back into the same structure before the next poll. Phase 1 implements only `HOLA_REQUEST_SNAPSHOT`; the remaining opcodes are reserved.

16.1.5 Synchronization Rules

- **Single producer:** the VM is the sole writer to the ring; multiple producers must funnel through the runtime shim.
- **Single consumer:** Phase 1 assumes exactly one analyzer; multi-consumer support will require an indirection table in the summary region.
- **Memory ordering:** producers write payload, then header, then bump `produce_seq`; consumers read `produce_seq` before pulling events and bump `consume_seq` after advancing `read_offset`.

- **Failure semantics:** when `dropped_events` increases, the consumer treats the last coherent boundary as unknown and resynchronizes from the first complete record after the drop.

16.1.6 Artefacts

The runtime implementation lives in `include/physics_runtime.h` and `src/physics_runtime.c` (`physics_runtime_init`, `physics_host_snapshot`, `physics_analytics_publish_event`). The formal model linkage is in `Physics_StateMachine.thy` and `Physics_Observation.thy`, which capture the observation-window semantics this protocol assumes. Future revisions will document multi-node governance flows and captable wiring for L4Re tasks once Phase 2 moves the analyzer into a dedicated microkernel service.

Chapter 17

Formal Verification

17.1 VM Formalization Roadmap

This roadmap records the staged approach to formally proving the StarForth virtual machine in Isabelle/HOL. It grows alongside the `.thy` theory files in the formal proof tree and serves as the architecture map for VM-level verification.

17.1.1 Objectives

The plan pursues three goals. It preserves room for architectural refactoring while the proofs evolve. It treats the physics-metadata proofs — `Physics_StateMachine` and `Physics_Observation` — as the first pillar, onto which additional modules snap within the same Isabelle session. And it keeps proof obligations discoverable from the `C` headers, documenting invariants close to the runtime code they constrain.

17.1.2 Session Layout

The verification workspace is organized as a layered set of theories. The lower layers are scaffolded and build independently of the physics layer; the physics theories remain fluid; the upper layers are planned.

Theory	Purpose (current status)
<code>VM_Core</code>	VM state record, pointer accounting, control-flag helpers (scaffolding).
<code>VM_Stacks</code>	List-backed stack model locale and depth lemmas (scaffolding).
<code>VM_Words</code>	Abstract wrappers for <code>>R/R></code> transfers (scaffolding).
<code>VM_Register</code>	Dictionary model for <code>register_word</code> (scaffolding).
<code>Physics_StateMachine</code>	Ring-buffer and mailbox invariants for HOLA (fluid).
<code>Physics_Observation</code>	Per-word physics updates, frame composition (fluid).
<code>VM_Dictionary</code>	<i>Planned:</i> dictionary lifecycle, physics wiring.
<code>VM_Interpreter</code>	<i>Planned:</i> interpreter state machine, semantics.
<code>VM_IO</code>	<i>Planned:</i> host capability shims, scheduler obligations.

The `VM_Formal` session captures `VM_Core` through `VM_Register` and builds without the physics layer. `Physics_Formal` extends it, keeping the observation machinery available while acknowledging that those theories remain fluid. Each planned theory imports the prior layers so the whole workspace stays incrementally checkable via `isabelle build`.

17.1.3 Development Guidelines

- **Pair source and proof** — when a C module gains an invariant, add or update the matching theory and cross-reference it in file-level comments.
- **Prefer locales** — define subsystem properties as locales and extend them in follow-up theories rather than rewriting existing ones.
- **Document assumptions** — every lemma states the runtime assumptions it relies on, such as stack-depth bounds or profiler hooks.
- **Keep proofs fast** — favor algebraic lemmas over brute-force tactics so developers can replay the session during normal builds.
- **Track TODOs** — leave `text` blocks or `FIXME` notes in the theory files marking which invariants still need encoding.

17.1.4 Next Actions

The immediate agenda is to land a green `VM_Formal` build before layering physics obligations; flesh out `VM_Core` with stack-depth and dictionary invariants once the C helpers stabilize; introduce a `VM_Stacks` theory capturing data and return-stack well-formedness and pointer safety (aligned with `STRICT_PTR`); model dictionary entries against `DictEntry` and connect the physics-metadata lemmas to the compiler pipeline; decide the C-to-Isabelle modeling path (AutoCorres versus manual abstraction) before tackling the interpreter loop; and wire a Makefile target to `isabelle build` so CI can gate on proofs as they expand.

17.1.5 Multi-Day Proof Checklist

The longer-horizon checklist proceeds in stages. A **session baseline** records the theory order in the project `ROOT` and snapshots current failures. **VM core invariants** expand `VM_Core` with pointer and limit invariants and error-flag rules, each phrased as a lemma inside the `vm_core` locale for reuse. **Stack model helpers** add peek/push/pop transfer lemmas for each stack direction, proving both success and guard-failure variants alongside runtime versions that expose the `dsp/rsp` relationship. **Word semantic wrappers** split per-module theories that import `VM_Words` and derive success and no-op lemmas in one or two lines. **Dictionary registration** centralizes lemmas in `VM_Register`, guarding reserved names. A **module-march template** repeats, for each of the roughly nineteen word modules, the cycle of identifying the C helper, confirming an Isabelle helper, creating the wrapper lemma, registering the word, and rerunning the session, with progress tracked in a status table. **IO and external hooks** stub locale assumptions for side-effecting words, and **housekeeping** rebuilds the session nightly, treating a green build as the commit gate. The **stretch goal** is a consolidated theorem: that all registered words satisfy their claimed semantics, combining the lookup lemmas with the per-word proofs.

17.1.6 VM-First Verification Roadmap

The proof order works upward from primitives. It begins with the **stack primitives** (`vm_push`, `vm_pop`, `vm_rpush`, `vm_rpop`), proving they refine the abstract transfers and raise `vm_error_active` precisely on overflow and underflow. It then proves the **return-stack words**, the **data-stack transfer words** (`DROP`, `DUP`, `SWAP`), the **arithmetic and logic modules**, the **dictionary and lookup words** (`CREATE`, `FIND`, `IMMEDIATE`), the **control-flow words**, the **compiler and interpreter infrastructure** (proving the interpreter halts on `vm_error_active` and honors abort and exit flags), and finally the **system and I/O words**, stubbing the required capabilities.

17.1.7 Status Snapshot

Stack and return-stack semantics are complete: `vm_push_sem`, `vm_pop_sem`, `vm_rpush_sem`, and `vm_rpop_sem` carry success, overflow, and underflow lemmas in `VM_StackRuntime.thy`. The parameter-stack words `DROP`, `DUP`, `?DUP`, `SWAP`, `OVER`, `ROT`, `-ROT`, `DEPTH`, `PICK`, and `ROLL` have proven abstract semantics, runtime correspondence, and wired dictionary registration. The return-stack words `>R`, `R>`, and `R@` have established abstract semantics and runtime equality. The model now exposes `cell_to_int/int_to_cell` conversions, and the `VM_Formal` session builds cleanly with the theories ordered `VM_Core` $\rightarrow$ `VM_Stacks` $\rightarrow$ `VM_StackRuntime` $\rightarrow$ `VM_DataStack_Words` $\rightarrow$ `VM_ReturnStack_Words` $\rightarrow$ `VM_Words` $\rightarrow$ `VM_Register`.

The next milestones introduce an instruction-pointer and memory abstraction so `(BRANCH)` and `(OBRANCH)` can be specified without referencing C internals; model loop frames on the return stack (`?DO`, `DO`, `LOOP`, `+LOOP`, `LEAVE`, `I`, `J`); formalize the compile-time control-flow stack so `IF`, `ELSE`, and `WHILE` can be reasoned about at registration time; and then extend the dictionary lemmas toward the arithmetic and logical modules.

Achieving these steps positions the project to claim that StarForth is, to the authors' knowledge, the first open-source FORTH VM accompanied by a full proof corpus of its core primitives in Isabelle/HOL, enabling proof-carrying reference builds and machine-verified stack discipline.

Appendix A

Physics Design Notes

A.1 The Cone of Influence

The cone of influence is a feedback-control architecture for the physics engine. It is named for a geometric metaphor used purely as a modeling device: the region of possible execution futures is treated as a cone that narrows over time. As the runtime accumulates observations, asynchronous and unpredictable publish/subscribe messaging is progressively constrained into deterministic, ordered information flow.

The driving idea is a closed loop. Observables from the early phase inform statistical models. Those models adjust runtime knobs. The knobs change behavior. The changed behavior generates new observables. Iterated, the loop moves the system from high-uncertainty asynchronous communication toward low-uncertainty deterministic ordering.

A.1.1 The Model

The cone occupies a conceptual three-axis space: a time axis (past to future), a scope axis (narrow word-level interactions to wide global scope), and a determinism axis (asynchronous fire-and-forget, through semi-deterministic retry-with-hints, to deterministic causal ordering). The cone narrows along three dimensions simultaneously:

- **Temporal** — past observations constrain present behavior.
- **Spatial** — an initially global scope resolves to specific word interactions.
- **Modal** — asynchronous uncertainty resolves to deterministic guarantees.

A.1.2 Three Operational Zones

Zone 1 — Wide and Async (Discovery). During the first N observations the system bootstraps. Many execution paths remain possible, entropy variance is high, latency is unpredictable, and scope is global. Publish events fire-and-forget into a ring buffer with no ordering guarantees. The goal is simply to discover which topics words publish to. Overhead is minimal, on the order of 200 ns. Observables available: entropy changes, temperature jumps, host snapshots, error rates.

Zone 2 — Narrowing and Semi-Deterministic (Learning). After M observations, entropy variance stabilizes, temperature reaches equilibrium, and the latency distribution narrows. Scope contracts to module level. Publish events now carry causal ordering hints: each message is tagged with a Lamport clock and a likely-destination set, and related messages are batched. The goal is to learn the dependency graph and subscriber patterns. Overhead rises to roughly

250 ns. New observables: entropy slope, temperature stability, latency percentiles (p95, p99), and the causal dependency graph.

Zone 3 — Narrow and Deterministic (Enforcement). In steady state, entropy plateaus, temperature is bounded by throttling, and latency carries guarantees against a service-level objective (SLO). Scope is specific: a word sends only to known targets. Publish events are ordered deterministically — buffered and sorted by causal order number — and a batch deadline of at most T ms is enforced. The p99 latency SLO is guaranteed, with backpressure applied on violation, and a subscriber whitelist eliminates surprises. Overhead settles at roughly 300 ns. The full observable set adds latency variance, thermal slope, and execution-pattern periodicity.

A.1.3 Metrics Inventory

The engine already captures a substantial baseline. Per-word metrics include `DictEntry.entropy` (raw execution count), and within `DictEntry.physics`: `temperature_q8` (EMA hotness), `last_active_ns` (recency), `mass_bytes` (footprint), `avg_latency_ns` (rolling average), and `state_flags`. Two fields, `acl_hint` and `pubsub_mask`, are reserved stubs.

Host-level snapshots feed the analytics heap from real POSIX and procfs sources:

Signal	Source	Status
<code>monotonic_time_ns</code>	<code>clock_gettime(CLOCK_MONOTONIC)</code>	Real
<code>cpu_count</code>	<code>sysconf(_SC_NPROCESSORS_ONLN)</code>	Real
<code>scheduler_policy</code>	<code>sched_getscheduler(0)</code>	Real
<code>scheduler_quantum_ns</code>	<code>sched_rr_get_interval()</code>	Real
<code>load_avg_milli</code>	<code>getloadavg()</code>	Real
<code>psi_cpu/io/mem*_milli</code>	<code>/proc/pressure/*</code>	Real (4.20+)
<code>cpu_total/idle_jiffies</code>	<code>/proc/stat</code>	Real
<code>cgroup_cpu_usage_us</code>	<code>/sys/fs/cgroup/cpu.stat</code>	Real (cgroup v2)
<code>cgroup_memory_current</code>	<code>/sys/fs/cgroup/memory.current</code>	Real (cgroup v2)

Profiler data (call frequency, total time, latency distribution, memory read/write patterns) and test-infrastructure outcomes round out the current set.

Several signals are not yet captured but are reachable: stack and return-stack depth at execution, dictionary lookup latency, block I/O queue depth, the data-flow graph of word-to-word calls, and the error rate. Useful ML features remain on the horizon: execution-pattern history, latency percentiles, entropy and thermal slopes, instruction-count estimates, and cache-miss rates where performance counters exist. Some quantities are out of reach by definition — future execution patterns require prediction, optimal scheduling is NP-hard, and ground-truth causality runs into observer effects.

A.1.4 The Control Layer: Runtime Knobs

Per-word knobs include execution priority (tuned by temperature), cache affinity (by entropy slope), temperature target, entropy half-life (cooling rate), latency limit (the SLO), stack-depth limit (recursion guard), and ACL level. System-level knobs include sampling cadence, analytics-heap size, profiler mode, pub/sub batch size and timeout, temperature smoothing alpha, and entropy decay rate. Each knob is driven by a specific tuning signal — for example, the latency limit responds to p99 measurements, and the sampling cadence responds to latency variance.

A.1.5 The Pub/Sub Gateway: Three Phases

The gateway is where words publish observations and where the control system acts. Its behavior tracks the three zones.

In **Discovery**, a word calls `physics_analytics_publish_event(topic, payload, size)` and the event is hash-inserted into the ring buffer; consumers poll when ready, with no ordering.

```
1 // Zone 1: fire-and-forget
2 physics_analytics_publish_event(PUBSUB_TOPIC_X, payload, size);
```

In **Learning**, the gateway identifies topic dependencies, learns subscriber sets, stamps each message with a Lamport clock, and tags it with a subscriber hint.

```
1 // Zone 2: learning phase
2 message.causal_order_num = ++g_causal_clock;
3 message.subscriber_hint = learned_targets; // "probably {WordY,
      WordZ}"
4 physics_analytics_publish_event(PUBSUB_TOPIC_X, message, sizeof(
      message));
```

In **Enforcement**, causal ordering is enforced by buffering and sorting on the order number; a batch deadline flushes pending messages; the p99 SLO is verified; delivery is restricted to a whitelist; and a full queue triggers backpressure on the publisher.

```
1 // Zone 3: enforcement phase
2 message.causal_order_num = ++g_causal_clock;
3 message.subscriber_targets = learned_whitelist; // exact set, no
      surprises
4 message.deadline_ns = now_ns + SLO_LIMIT_NS;
```

A.1.6 The Feedback Loop

The control law maps observed state, through a detection rule, to a knob adjustment. A very hot word with rising entropy lowers execution priority, shortens the entropy half-life, tightens the stack limit, and raises the sampling cadence. A p99 latency that persistently exceeds the SLO tightens the stack-depth limit, the batch timeout, and the batch size. A reached entropy plateau — low slope, low variance — raises cache affinity, the temperature target, and execution priority. Excessive latency variance raises sampling cadence and profiler detail. An error-rate spike restricts the ACL level and lowers priority. Scheduler noise from core migration raises cadence and pins affinity.

A hot-word emergency illustrates the loop end to end: a word saturates at `0xFFFF` with a climbing entropy slope; the controller lowers OS priority, accelerates cooling, caps recursion, raises sampling, and isolates the word via the ACL level; the next cycle checks whether the temperature stabilized, restoring normal priority on success or escalating to throttling and governance alert on failure; offline analysis later mines the event for better detection heuristics.

A.1.7 ML and Statistical Feedback (Future)

Once the three zones operate and history accumulates, the design anticipates four further phases: predictive models for latency, hotness, and optimal batch size; anomaly detection for execution-pattern shifts, phase changes, SLO violations, and early thermal runaway; optimization of core affinity, batching strategy, priority ordering, and profiler mode; and governance integration with machine-checked proofs in Isabelle — observable invariants, a cone-narrowing convergence theorem, verified control loops, and proven SLO bounds.

A.1.8 Open Design Questions

The design records several unresolved choices, each with a working recommendation:

- **Zone transition** — time-based, confidence-based, or hybrid. Recommendation: hybrid (start time-based, add confidence).
- **Ordering strength** — causal only, total, or causal-within-module and total-globally. Recommendation: hybrid.
- **SLO scope** — global, per-word, or governance-derived per category. Recommendation: derived.
- **Loop style** — open-loop, closed-loop, or hybrid. Recommendation: hybrid, with closed-loop reserved for critical knobs.
- **ML training** — in-system, offline, or hybrid. Recommendation: hybrid, with online anomaly response and offline refinement.

A.1.9 Glossary

- **Causal ordering** — messages ordered by dependency ($A \rightarrow B$ if A must precede B).
- **Lamport clock** — a scalar clock that respects causality.
- **Cone of influence** — the region of possible execution futures, narrowing as confidence grows.
- **Zone** — an operating region (async discovery, semi-deterministic learning, deterministic enforcement).
- **Knob** — a runtime-adjustable parameter affecting behavior.
- **SLO** — service-level objective (a latency guarantee).
- **Backpressure** — flow control that slows publishers when a queue fills.
- **Entropy** — execution count per word, an implicit hotness measure.
- **Temperature** — the exponential moving average of entropy, an explicit hotness measure.
- **Half-life** — the time for temperature to decay to half its peak.

A.2 From Observability to Control

The physics engine answers the question of what the system is doing. Scheduling and memory management answer the question of what it should do differently. The physics engine supplies observability — entropy slope, temperature trend, p99 latency. Scheduling and memory management supply control: execution placement and data placement respectively, joined by feedback-control loops.

Three subsystems consume physics signals. The scheduling system governs word priority, core affinity, preemption hints, latency SLOs, and throttling. The memory-management system governs cache tier, block placement, garbage-collection triggers, and eviction policy. Between them, feedback loops close observation onto action.

A.2.1 The Scheduling System

StarForth executes on top of a host scheduler (L4Re or Linux). The VM runs largely on a single thread, cannot directly reorder word execution within a definition, and influences the host only through priority hints (`setpriority`, `sched_setparam`); the return stack dictates nesting order.

Within these limits, physics can still control scheduling. At the OS level it sets per-word priority (hotter words receive lower nice values), CPU affinity (hot, cache-sensitive words pinned together; interfering words separated), and preemption hints (hot, rapidly growing words made non-preemptible for batching).

```

1  typedef struct {
2      int os_priority;           // nice level or sched_priority
3      int scheduling_class;     // SCHED_OTHER, SCHED_FIFO, SCHED_RR
4      int cpu_affinity_mask;    // which cores allowed
5  } sched_hint_t;
6
7  // hotter = lower nice = higher priority
8  word->sched_hint.os_priority = (word->temperature_q8 > 0x8000) ? 5 :
    10;

```

Even without host control, a VM-level scheduler can respect physics. Inside `execute_colon_word()`, hot words on a critical path can be executed with affinity or batched with related words, while cold words are deferred or batched for idle time. Adaptive batching groups words that physics observes always run together — IF/THEN, for instance — to minimize cache misses.

A latency-SLO mechanism times each execution against a per-word limit and, on violation, adjusts knobs: it boosts priority, lowers the stack-depth limit, and shortens the batch timeout, logging the breach.

A.2.2 The Memory-Management System

StarForth uses a single 5 MB dictionary heap (statically allocated, no fragmentation), block storage of 1024 blocks of 1024 bytes, and two 1024-cell stacks. There is no explicit memory manager; allocation is implicit in the dictionary.

Physics can drive placement across a logical memory hierarchy. A tier is chosen from word temperature and entropy slope:

```

1  typedef enum {
2      MEM_TIER_L1, MEM_TIER_L2, MEM_TIER_L3,
3      MEM_TIER_DRAM, MEM_TIER_BLOCK
4  } mem_tier_t;
5
6  mem_tier_t choose_tier(DictEntry *word) {
7      if (word->physics.temperature_q8 > 0xE000) return MEM_TIER_L1;
8      if (word->physics.temperature_q8 > 0x8000) return MEM_TIER_L2;
9      if (word->physics.temperature_q8 > 0x4000) return MEM_TIER_L3;
10     if (word->physics.entropy_slope > 0)         return MEM_TIER_DRAM;
11     return MEM_TIER_BLOCK; // cold: evict to block storage
12 }

```

Cache coloring uses the learned call graph: words that frequently co-execute are placed in different cache sets to avoid conflict. Garbage collection is triggered not by a simple capacity threshold but by memory pressure (drawn from the host PSI snapshot) combined with the presence of cold, stale words to reclaim. Eviction replaces pure LRU with a temperature-weighted score — age multiplied by inverse temperature — so a candidate is hot only when both old and cold:

```

1  uint64_t score = (age / 1000000) * ((0xFFFF - temperature_q8) + 1);

```

```
2 | // older + colder => higher score => evicted first
```

Block storage absorbs the coldest words. A word spills to disk when it is cold, has not run for several minutes, sits outside any SLO-critical path, and memory pressure is high; it is lazily reloaded on next use, with its physics re-touched on access.

A.2.3 Integration Points

Physics observes (entropy, temperature, recency, latency, call graph, stack depth, error rate, memory pressure) and feeds the scheduler, which decides priority, affinity, batch group, preemption, SLO, throttling, and stack limits. Execution respects those hints and generates new metrics, which feed the memory manager’s decisions about cache tier, block placement, GC trigger, eviction, spilling, and prefetch. Placement then affects the next iteration’s execution, closing the loop.

Two narrow APIs connect the components. The physics-to-scheduling interface (`sched_control_t`) carries priority, affinity mask, batch group, preemptibility, latency limit, and stack limit; `sched_update_word()` applies adjustments and `sched_report_execution()` reports observed latency. The physics-to-memory interface (`mem_control_t`) carries preferred tier, cache-set preference, spill-age threshold, evictability, and eviction priority. Scheduler and memory manager also coordinate directly: a batch decision notifies the memory manager so it can cache-color the group, and an eviction of a critical-path word warns the scheduler before disabling that word’s fast path.

A.2.4 Implementation Phases

- **Phase 1 — Instrumentation (current).** Physics observes and publishes metrics; scheduling and memory read them advisably.
- **Phase 2 — Weak Control.** Scheduling adjusts OS priority hints; memory becomes cache-tier aware; still advisory.
- **Phase 3 — Strong Control.** Scheduling enforces the latency SLO; memory enforces GC and eviction; the feedback loop is closed.
- **Phase 4 — ML Integration.** A model predicts latency, memory, and scheduling outcomes; governance approves knob adjustments.

A.2.5 Worked Examples

Hot word going critical. An I/O word heats during a file scan. Physics reports rising temperature and entropy slope and alerts both subsystems. Scheduling boosts the word’s priority, enables I/O batching, caps recursion, and sets a 10 μ s SLO. Memory promotes the word to L2, prefetches related words (`BLOCK_WRITE`, `BUFFER`, `UPDATE`), and protects it from eviction. Execution then meets the SLO at p99 $\approx$ 8.5 μ s while the temperature stabilizes, after which both subsystems can act more aggressively on the now-learned pattern.

Cold word emergency eviction. Under 95% dictionary use and 96% cgroup memory, GC triggers. Physics scores eviction candidates by age times inverse temperature, selecting the coldest, oldest word first while protecting a recent hot word on the critical path. The chosen word spills to block storage, is marked `WORD_SPILLED`, and reclaims space. Much later, when the user invokes it again, the executor detects the flag, reloads it from storage in roughly 50 μ s, runs it, and lets physics re-learn its pattern.

A.2.6 Critical Design Decisions

- **Scheduling granularity** — per-word priority, coarse word batches, or a hybrid that starts fine-grained and learns optimal batches. Recommendation: hybrid.
- **Memory placement accuracy** — static placement, dynamic migration, OS-hint delegation, or logical tiers with no physical movement. Recommendation: logical tiers as scheduling hints.
- **GC versus spilling** — always compact, always archive, or a governance-driven per-word policy. Recommendation: governance-driven.

A.2.7 Open Questions

The design leaves several questions for resolution: whether the VM should implement its own scheduler or defer entirely to the host; whether dictionary words can be physically relocated without breaking pointers, or only hinted logically; whether block storage serves persistence or active-memory overflow; how scheduling and memory decisions should interact with L4Re IPC deadlines and capability delegation; who owns eviction and spilling policy; and whether the added control overhead (roughly 100 ns atop the existing 200–300 ns per event) is acceptable for target latencies.

A.3 Physics Signal Inventory

This inventory enumerates the telemetry the physics-driven scheduler can sense today, drawn from two sources: the L4Re microkernel (through the StarshipOS loader stack) and the in-process StarForth VM. It defines the input stage of the scheduler. The guiding metaphor is an operational amplifier: the positive input ties to real host signals, the negative input ties to VM-internal counters, and a Bayesian inference loop stabilizes the gain. The metaphor is a modeling device for the control architecture, not a literal circuit.

A.3.1 L4Re and Microkernel Signals

Kernel Interface Page (KIP). The KIP exposes a high-resolution monotonic clock (`l4_kip_clock_ns()`) for timing when no RTC is present, a kernel build fingerprint for correlating inference data with kernel revisions, the scheduling granularity (an upper bound on how fine-grained the physics scheduler can react), physical memory descriptors (how much RAM can be earmarked for in-RAM inference buffers), and platform/SMP information. SMP availability doubles as a noise hook: work can be scheduled on sibling cores to inject controlled jitter and keep the model honest.

Scheduler object. `l4_scheduler_info()` reports the online CPU bitmap and maximum CPU count, bounding how many physics loops run in parallel. `l4_sched_param_t` (priority and quantum) records the loader’s configured timeslice as a prior per VM. The scheduler-class flags reveal whether fixed-priority or weighted-fair-queue scheduling is active, which shapes how aggressively words may be heated or cooled. `l4_sched_cpu_set()` pins the main or physics worker thread to a core subset during experiments.

Loader and environment capabilities. The loader script (`quark.lua`) wires named capabilities the runtime can tap: `vbus_storage` for block-device enumeration and driver statistics (queue depth, error counts); `ahci_chan` for completion-interrupt rates and per-port error registers; `rtc_ns` for wall-clock sync; `cons_mux` for console input cadence as a proxy for interactive

heat; `L4.Env.log` for scheduler and hardware warnings that should bias temperature decay; and `L4.Env.vesa` for framebuffer refresh/blit latency.

Further kernel APIs. `l4_thread_control()` and `l4_thread_ex_regs()` expose per-thread UTCB pointers and instruction pointers for correlating VM stalls with kernel scheduling; `l4_ipc_error()` surfaces IPC failure codes that could feed entropy penalties for fault-prone words; and virtual IRQ devices can be counted to derive environmental noise injected into the VM.

A.3.2 StarForth VM Signals

Core runtime state. `DictEntry.entropy` is the per-word execution counter, the baseline for macro temperature. `DictEntry.physics` holds the tunable knobs (temperature, last-active timestamp, mass). The data and return stack pointers (`VM.dsp`, `VM.rsp`) report stack depth, whose steep excursions imply turbulent execution and feed variance estimates. Mode and state fields reveal interpret-versus-compile state and the currently executing word. The `error` and `halted` flags are binary fault signals that spike entropy decay when they flip. Log volume and severity mix are themselves telemetry: heavy warnings can cool risky words.

Profiler and instrumentation. `WordProfile` supplies per-word total time, call count, and min/avg/max latency — a direct feed for `avg_latency_ns` and Bayesian likelihoods. `MemoryProfile` supplies read/write counts and bytes as a mass-energy proxy for memory-thrashing words. The profiler counters (VM cycles, dictionary lookups, stack ops, allocations) seed default temperatures and heat capacities. `profiler_word_count()` stays available even without detailed profiling, and `vm_debug_dump_state()` provides a structured post-mortem the Bayesian tool can parse to reset priors after a crash.

Block and storage subsystem. Block-subsystem globals report working-set size, dirty-block counts, and block-allocation-map churn. `blkio_info()` exposes device geometry and the read-only bit, informing whether a cooling word should migrate to RAM or disk tiers. `blkio_read/write` return codes give immediate error feedback. Cache slots track hit/miss rate and write-back frequency to infer subsystem momentum.

A.3.3 Op-Amp Signal Flow

The signal flow follows the amplifier metaphor in five stages:

1. **Positive input (microkernel)** — real-world noise: CPU availability, I/O latency, RTC drift, IRQ storms.
2. **Negative input (VM)** — internal state: entropy, latency, stack tension, storage dirty set.
3. **Amplifier** — the Bayesian inference loop adjusts priors and updates word physics.
4. **Output** — updated `DictPhysics` structs and scheduler hints that modulate execution order and block placement.
5. **Feedback** — the observation window width is adjusted by variance (a gauge study) and the cycle repeats.

A.3.4 Messaging and IPC Considerations

The pub/sub backbone is shared-memory-first: a ring buffer with sequence counters inside a dedicated analytics heap (10 MiB by default). Producers write events, flip a counter, and continue, with no blocking semantics inside the VM. On L4Re, where IPC is synchronous, IPC serves only as a notification channel — a publisher pokes a notification thread that drains the ring and forwards to subscribers; virtual IRQs offer an alternative non-blocking wakeup. On Linux the same API is backed by condition variables or eventfd behind a common shim, keeping the VM path identical across platforms. All state stays resident in a fixed-size analytics heap with no dynamic expansion. That heap is separate from the 5 MiB VM arena (`VM_MEMORY_SIZE`), so the dictionary, stacks, and block-subsystem budgets remain untouched.

A.3.5 Host Snapshot Shim and Analytics Heap

Phase 1 ships a concrete implementation. `physics_runtime_init()` reserves the analytics heap (10 MiB default) whose layout the HOLA contract documents, publishing the header and region descriptors HOLA consumes. `physics_host_snapshot()` abstracts POSIX and L4Re scheduler probes and feeds ring-buffer events on channel `0x00000002` via `physics_analytics_publish_event()`. The heap header, event records, and mailbox schema live in `include/physics_runtime.h`; the runtime lives in `src/physics_runtime.c`. The interface is deliberately ABI-stable so governance tooling can mirror it without pulling in C sources. On the POSIX path, Phase 1 additionally captures Linux PSI values mapped into `psi*_avg{10,60,300}_milli`, `/proc/stat` total and idle jiffies, and cgroup v2 CPU and memory usage where available; flag bits advertise which sources were populated.

A.3.6 Conventions and Constraints

Several conventions govern the inference loop. The observation window combines a time-based heartbeat with event-count triggers: events act as “excitement” that boosts entropy, while publish and decay operations cool at roughly half that rate (tunable). All computation uses 64-bit fixed-point integers. Physics snapshots can optionally be persisted into FORTH block storage and reloaded during (INIT) to simulate a warm boot or replay a training sequence. Descriptor inheritance flows from module, vocabulary, and VM defaults down to individual words, so sensible priors seed at multiple levels; every tier exposes the same attribute schema — temperature, latency, mass, state flags, ACL hints, pub/sub mask, and pinned flag. VM-level rollups mirror per-word metrics and define operating bands (COLD, WARM, HOT, CRITICAL) that governance and the scheduler shim can respond to. Isabelle captures the formal state machine, invariants, and IPC-handshake proofs; HOLA defines the shared-memory layout and control protocol consumed by both the VM and external analyzers.

A primitive seed table (`physics_metadata_apply_seed()`) installs initial priors for high-impact primitives — control flow, I/O, block subsystem, save-system — so temperature and latency estimates do not start at absolute zero; governance tooling can extend or override the table once the Bayesian loop is in place.

A.3.7 Immediate Research Tasks

1. Prototype a thin KIP/scheduler shim exposing the listed signals to userland C with no filesystem: on L4Re via `l4re_kip()`, on POSIX via a stub that feeds monotonic time and scheduler defaults.
2. Inventory the IO-server (vbus) protocol to pull queue-depth and error counters for AHCI, NVMe, and virtio backends.

3. Define the shared-memory layout between the VM and the Bayesian analyzer, defaulting to a 10 MiB heap (header, ~6 MiB event ring, ~3 MiB summary/scratch, padding), honoring the platform split between POSIX and L4Re messaging.
4. Extend the profiler to snapshot `MemoryProfile` deltas without enabling full verbose mode, keeping overhead low.
5. Derive initial priors for key primitives (control, I/O, block) from handcrafted knowledge plus the loader's priority and quantum configuration.

Appendix B

Messaging Architecture

B.1 The StarshipOS Messaging Field

The StarshipOS messaging subsystem is modeled as a thermodynamic field rather than a passive bus. In this framework, message propagation and processing behave as a physical system: discrete messages behave analogously to quanta (photons), and words — the fundamental executable units of the StarshipOS runtime — behave as particles, or loci of interaction. The thermodynamic vocabulary is a modeling tool for message dynamics; it grounds prioritization and scheduling, credit allocation and flow control, thermal regulation of load, entropy-driven policy and ML feedback, and instrumentation of message flow.

B.1.1 Conceptual Model

The message field. The field is the logical substrate mediating all communication. It comprises endpoints (boundaries between word spaces and the field), channels (directed pathways: virtual channels, pub/sub topics, RPC routes), and field properties (dynamic parameters such as message density, propagation latency, and entropy). The field is not centralized; it emerges from the coordinated behavior of the broker/router (`sf_msg-srv`), the ABI provider (`sf_msg-drv`), and the participant library (`libsfmsg`) over shared memory rings and control IPC.

Messages as quanta. Messages are modeled as energy quanta with four core properties:

Property	Description
Energy	Encodes priority, TTL, QoS, and thermodynamic state
Momentum	Routing vector through the field
Cross section	Probability of absorption by a word
Entropy	Thermodynamic measure of the flow's state space

Messages may be absorbed, emitted, or scattered by words and services, by analogy with photon-atom interactions.

Words as particles. Each word is a particle in the field — a locus of interaction rather than an isolated routine — with intrinsic mass (computational cost), charge (degree of side-effect or state mutation), and cross-section (the message types to which it responds). When a message reaches a word, the interaction may excite the word (trigger execution), alter its state, or pass through unabsorbed; executed words may emit new messages, either deterministically or in a burst (stimulated emission).

B.1.2 Mathematical Formulation

Maxwellian entropy. The system adopts a Maxwell–Boltzmann entropy model, treating each endpoint or channel as a thermodynamic micro-system. For a given flow f , the flow entropy $S_f(t)$ is a scaled function of the variance of inter-arrival intervals $\sigma_t^2(f)$, the normalized queue occupancy $Q(f) \in [0, 1]$, the credit-utilization ratio $U(f)$ (credits consumed over granted), the delivery fan-out $R(f)$, and the normalized message TTL $T(f)$, with scaling constant k (typically 1):

$$S_f(t) = k \cdot g(\sigma_t^2(f), Q(f), U(f), R(f), T(f)) \quad (\text{B.1})$$

High entropy indicates unpredictable, high-energy flows approaching saturation; low entropy indicates stable, structured behavior.

System temperature and pressure. Temperature Θ is the density-weighted average energy of messages in the field. Pressure Π represents backlog intensity, aggregated over flows from queue occupancy and credit utilization:

$$\Pi = \sum_f Q(f) U(f) \quad (\text{B.2})$$

Aggregate system entropy Σ sums the per-flow entropies:

$$\Sigma = \sum_f S_f \quad (\text{B.3})$$

Together these quantities give real-time metrics of system health, suitable as inputs to control algorithms.

B.1.3 Header and State Extensions

Messages carry thermodynamic metadata alongside conventional routing and control information. A per-message header extension encodes a composite energy scalar derived from QoS class, TTL, and policy weighting; a Maxwellian entropy estimate computed at the sender or broker; and an optional producer-side temperature hint for expected burstiness:

```

1 struct sfm_hdr_thermo {
2     uint16_t energy_q8;      // priority/TTL/entropy composite (Q8.8)
3     uint16_t entropy_mx;    // Maxwellian entropy (Q8.8)
4     uint16_t temp_hint;     // optional producer temperature hint
5     uint16_t reserved;
6 };

```

Endpoints maintain continuously updated thermodynamic state — current entropy, temperature, pressure, credit utilization, and a last-refresh timestamp:

```

1 struct sfm_endpoint_state {
2     float    entropy_current;
3     float    temperature;
4     float    pressure;
5     float    credit_utilization;
6     uint64_t last_refresh_tsc;
7 };

```

B.1.4 Scheduler and Credit Integration

Credits as energy quanta. Credits represent the available energy budget for message emission along a channel. High-entropy flows receive smaller, more frequent credit refreshes for tight regulation; low-entropy flows receive larger, batched credits for looser regulation. The recycling policy — immediate, piggyback, or batched — is selected dynamically from the flow’s entropy and temperature.

Thermodynamic scheduling. Scheduling priority is a function of QoS class, deadline, and flow entropy:

$$P_{\text{sched}} = f(\text{QoS}, \text{deadline}, S_f) \quad (\text{B.4})$$

Low-entropy real-time flows are scheduled deterministically; high-entropy bulk flows are throttled or coalesced; aging and anti-starvation mechanisms apply within entropy bands. This prevents high-entropy flows from destabilizing the system while letting low-entropy flows achieve predictable latency.

B.1.5 Monitoring and Control Loop

A control loop in `sf_msg-srv` maintains field stability in four steps:

1. **Sampling** — entropy, temperature, and pressure are sampled periodically from endpoints and channels.
2. **Aggregation** — system-wide Σ , Θ , and Π are computed.
3. **Policy evaluation** — control laws or ML bandits adjust credit windows, scheduling weights, and routing from the sampled state.
4. **Actuation** — the scheduler and credit allocator apply the updated parameters.

The loop runs with fixed thresholds or adaptive policies; ML components treat entropy as the order parameter for optimization.

B.1.6 Implications for the Runtime

Because words are already explicit entities with well-defined entry points, modeling them as particles interacting via message photons integrates cleanly with the existing FORTH execution model. The runtime gains a unified abstraction for computation and communication, fine-grained control over execution dynamics through entropy, a principled basis for prioritization in place of ad-hoc heuristics, and thermodynamic instrumentation for debugging and analysis.

B.1.7 Future Work

Four directions extend the model: a compact formal policy language for entropy-based routing and scheduling; distributed thermodynamic fields spanning multi-node topologies, where message photons propagate across network transports; entropy-driven garbage collection and memory tiering that couple message entropy with VM memory placement; and visualization tooling that renders message flow as a dynamic field.

B.1.8 Reference Header

A draft reference header, `include/sfm_thermo.h`, accompanies the model as design reference rather than production code (C99, released CC0 1.0 / public domain). It provides fixed-point helpers for Q8.8 and Q16.16 conversion; the `sfm_hdr_thermo` header extension; a per-flow state structure (`sfm_flow_state_t`) tracking Welford inter-arrival statistics, EWMA queue occupancy and credit utilization, fan-out and TTL trackers, and derived entropy, temperature, and pressure; a field snapshot structure; and a credit-decision structure. Its API initializes a flow with watermarks and entropy thresholds (`sfm_thermo_init_flow`), updates state on enqueue (`sfm_thermo_on_enq`), samples and decides a credit allocation (`sfm_thermo_sample_and_decide`), aggregates flows into a field snapshot (`sfm_thermo_aggregate`), and fills a message's thermodynamic header (`sfm_thermo_fill_hdr`).

B.1.9 References

- J. C. Maxwell, *Illustrations of the Dynamical Theory of Gases*, Phil. Mag., 1860.
- L. Boltzmann, *Weitere Studien über das Wärmegleichgewicht unter Gasmolekülen*, 1872.
- StarshipOS Internal Messaging Architecture Specifications.